

Chapter 1

Introduction

In this chapter, a general picture of the research problem is provided, its significance, and depth. Coconut disease-finding problems can be assisted with the help of learning. It discusses the objectives of the project, motives in doing it, and anticipated outcomes. It also provides an overview of the methodology used briefly and structure of the report in general.

1.1 Introduction

Coconut farming is a significant agricultural activity in most of the tropical regions, yet its yield is deteriorating due to several severe pathologies, including Bud Rot, Leaf Rot, Yellow Leaf Disease, and Stem Bleeding. The diseases may render the plants highly ill, reduce the yield of fruits, and cost the farmers money in the long run. Research indicates that the spread of coconut diseases is still rampant due to climatic variations, poor management of the diseases, and late diagnosis. This renders their identification at a tender age quite difficult in most countries in which the coconuts are cultivated [1], [3], [7].

The manual observation used in traditional diagnosis is subject to bias, inconsistency and depends on the presence of experts. Visual similarity between the symptoms of the disease and influence of the environmental factors, makes it even more difficult to find the right ones [4], [8]. Due to this reason, farmers fail to detect diseases early enough hence allowing infections to proliferate in plantations.

Another good option to automated detection of diseases by classification based on images is deep learning. ResNet, DenseNet, MobileNet and EfficientNet CNN architectures have demonstrated that they can detect agricultural diseases quite well since they are able to extract features quite well [10], [14], [16]. Recent research highlighted the importance of lightweight models to implement the model in the field and the need to explain AI to improve trust and interpretability in agricultural systems [20, 22].

In this paper, a deep learning-based approach to detect five major coconut infections by using four pretrained models; DenseNet121, ResNet50, MobileNetV2 and MobileNetV3 is proposed. The models are trained and tested on a 5,000 image set that is labeled. Grad-CAM is an explainable AI that is utilized to reveal disease-specific regions of leaf images. The idea is to develop an accurate disease detection system that can be deployed by various farmers and enable them to intervene early and produce coconuts over a long period.

1.2 Motivation

Coconut farming sustains millions of people yet outbreaks of diseases continue to make the economy suffer huge losses. The lack of resources among farmers in rural settings makes it difficult to get diagnoses of different diseases in time since farmers are unable to easily access agricultural professionals and it is very difficult to notice the first signs of diseases. The slow rate of detecting a disease may easily infect the whole plantations and this may negatively impact productivity in the long term. A decent alternative is deep learning that allows you to recognize diseases with pictures of leaves fast, reliable, and usually accurately. Such models as ResNet50 or DenseNet121 are able to identify very small visual patterns that are frequently overlooked by a human eye when one observes objects. Conversely, such lightweight architectures as MobileNetV2 and MobileNetV3 allow creating diagnostic tools that can be applied on mobile gadgets. The inclusion of explainable AI will bring an even more open system where everyone concerned will be able to understand why a prediction was made. This project will aim at producing a cheap and scalable coconut disease detection system which will satisfy the requirements of the tropical farming communities. It will also provide the researcher with an opportunity to know more about deep learning, computer vision, and the way to apply AI to real life.

1.3 Objectives

According to our study, we have the following main objectives:

- To develop a system built on deep learning that will be capable of identifying and classifying the most prevalent diseases of coconut trees.
- To apply what transfer learning can do to pretrained models (DenseNet121, ResNet50, MobileNetV2, MobileNetV3) to classify faster and more accurately.
- To test the models in terms of accuracy, precision, recall, F1-score, and confusion matrices.
- To implement explainable AI (Grad-CAM) to reveal disease-specific features.
- To use Grad-CAM to find out how easy it is to understand each model and see how well the highlighted areas reveal the real disease symptoms.
- To compare the effectiveness of the various models and determine the most appropriate way of constructing a model to detect coconut disease.
- To investigate whether or not lightweight models can be scaled and used in real time or on a mobile device.
- The proposed solution aims to provide an effective and economic solution that would assist in the timely diagnosis and reduce crop losses.
- To discover the performance of each deep learning architecture when used in real time or on a mobile device.

- To establish a performance comparison mechanism that identifies the optimistic architecture to field-level coconut disease detection that is feasible as well as scalable.

1.4 Research Questions

Here are three research questions derived from based on our research:

RQ1: How well can deep learning models use image-based data to tell the difference between coconut leaf diseases in real-world farming situations?

RQ2: What are the differences in performance, computational cost, and interpretability between the chosen models, and which architecture is best for use in the field?

RQ3: How well can explainable AI methods like Grad-CAM show disease-specific leaf areas and make coconut disease classification models easier to understand?

1.5 Project Outcome

The expected outcomes of this project include:

- An operational deep learning framework proficient in categorizing five principal coconut diseases.
- Performance benchmarks for DenseNet121, ResNet50, MobileNetV2, and MobileNetV3 using a number of different evaluation metrics.
- Explainable AI visualizations (Grad-CAM) that show important parts of an image that affect model predictions.
- A comparison that shows the pros and cons of both heavy and lightweight architectures.
- A possible basis for creating a disease detection tool that works in real time and is easy to use on mobile devices for farmers.
- A step toward smart agriculture by making it easier to find diseases, manage crops in a way that is good for the environment, and lower economic losses.

1.6 Organization of the Report

Chapter 1: The Beginning

In this chapter, the author discusses the background, the objectives, and the scope of the research and its importance. It also discusses the rationale behind conducting the study and how the deep learning models and explainable AI (X-AI) can be integrated into the disease-detecting model of agriculture.

Chapter 2: Background

The chapter is a literature review on the topic of the research. It discusses much on the detection of diseases in agriculture, the application of deep learning approaches, and where the models can be integrated into the proper processes.

Chapter 3: The Method of doing the Research.

This chapter describes the research design, research methods, and data collection. It also discusses the model architectures, training procedures, hyperparameters and the Grad-CAM technique and the way these steps have been applied to make the classifications more precise.

Chapter 4: Implementation and Results.

In this chapter, the author will describe how the system and hardware, software, and software tools required to train and test the models are set up and configured. The model performance results are presented and the models results are compared with one another in terms of accuracy, loss, precision, recall, and f1-score.

Chapter 5: The challenge of standards and design in engineering.

The engineering standards that will be discussed in this chapter include the hardware, software, and communication standards of the project. It also discusses the design issues that emerged during the creation of the models applicable to real-time disease detection and the necessity to balance the accuracy of the models and the hardware resources that would be required to operate and use the models in the low-resource places.

Chapter 6: Final Thoughts

The concluding chapter provides a summary of the research findings, criticizes the research done and presents possible future research areas to enhance and pursue the study. It also indicates the weaknesses of the system and how to rectify some of the flaws of the system in the subsequent models of the disease detection system.

Chapter 2

Background

This chapter provides the data required in the insight of the concepts, approaches, and past studies regarding the application of deep learning to locate coconut disease. It discusses fundamental concepts, reviews significant literature, provides an overview of related projects, identifies gaps in the research, and indicates how this project fits in the already conducted work in the field.

2.1 Introduction

In order to ensure agriculture can sustain, a search for plant diseases must be done early to ensure crops remain healthy. The coconut trees, which are widely cultivated in tropical countries, are highly prone to infections such as Bud Rot, Leaf Rot, Yellow Leaf Disease and Stem Bleeding. These diseases damage the plantations significantly, thereby reducing the volume of crops that they are able to produce and derailing the livelihoods of the farmers. When farming is done on a large scale, manual inspection as a diagnostic tool is both time-consuming, unreliable, and fallacious in traditional diagnosis. Deep learning has revolutionized the process of locating diseases in images in such a way that it can be used to automatically identify diseases with high accuracy and scalability. Models of transfer learning such as DenseNet, ResNet and MobileNet, and Convolutional Neural Networks (CNNs) have demonstrated that they can perform an excellent task in the identification of agricultural diseases. These models are capable of detecting small variations in the texture and color of infected leaves and this aspect makes them effective in the detection of diseases at an early stage. In this chapter, the author discusses the development of deep learning in the field of plant pathology, examines the existing research, and preconditions the proposed coconut disease detection system.

2.2 Literature Review

Deep learning has emerged as the major method applied to identify crop diseases automatically since it is capable of extracting significant information out of complex images of agriculture. Various researchers have applied convolutional neural networks (CNNs) and advanced deep learning models to detect diseases in the coconut leaves with promising results.

Subbaian et al. [1] have applied YOLOv4 to detect coconut leaf diseases and achieved significant results with an F1-score of 88 percent and an average of Precision (mAP) of 85 percent. This example demonstrates that modern object detection systems are able to handle changes in the lighting conditions and backgrounds. Singh et al. [2] developed a deep learning model that incorporated both custom CNN models and pretrained models such as

MobileNet and InceptionResNetV2. This framework could detect stem bleeding, Leaf blight, and Red Palm Weevil infestations with an 96.94 percent accuracy. In a study by Brar et al. [3], the ResNeXt50 architecture was used to classify Coconut Leaf Spot disease into six different levels of severity and it achieved an accuracy of 91.77, which proves to be an effective architecture of CNN in the detailed assessment of the disease.

The success of the model is also highly dependent on the data sets that are used to support it. Thite et al. [4] reported a uniform dataset of 5,798 images of coconut leaf diseases in five main categories thus enhancing replicable research and comparative model estimation. In other review papers, such as the one by Nivethitha et al. [5], it is discussed that increasingly more people are turning to CNN-based frameworks to discover plant and coconut diseases. They also identify issues in existing datasets and the necessity of congruence between various environmental settings.

Other researchers propose integrated models that combine some models of image processing and deep learning. Maray et al. [6] came up with the AIE-CTDDC model, which uses Bayesian fuzzy clustering as the separation criterion and integrates CapsNet and GRU as its classifiers. Their hybrid approach was more effective than the others, providing evidence to the fact that deep learning and the traditional segmentation can be effective together. Muthulakshmi et al. [7] compared the use of ResNet-50, EfficientNet-B0, and DenseNet-201 using various activation functions and discovered that the selection of an activation function can greatly influence the achieved performance of a model.

Even in old fashion image processing, it can be detected early. George et al. [8] achieved 80.36% pixel accuracy in detecting a stem bleeding disease by color thresholding compared to the traditional ones. But such techniques do not perform well when there is a change in lighting or background, which demonstrates the superiority of deep learning. Pests have also been located using deep learning, such as Kavithamani and UmaMaheswari [9] study to locate whiteflies on coconut leaves. This demonstrates the increased relevance of AI in pest control.

A number of ML-based methods are studied. GLCM and SIFT features were used alongside common classifiers by Karegowda et al. [10] and achieved accuracy of up to 100% with SIFT features though meticulous feature engineering was required. Wijethunga et al. [11] have come up with an extensive framework of integrating disease detection, environmental monitoring, and growth prediction to depict the flexibility of machine learning in agricultural decision support systems. The example of a system that integrates deep learning and XAI based on InceptionV3 is the Cocoscan by Santhi and Murugan [12]. This demonstrates that the automated diagnosis of coconut disease can be made more comprehensible.

Numerous deep learning systems have been very effective in the detection of coconut leaf disease. In the work of Anitha and Santhi [13], the DenseNet-121 achieved approximately 99 percent of accuracy and the study of Toradmalle et al. [14] demonstrated that both InceptionResNetV2 and DenseNet121 were both effective at identifying diseases at an early stage. Research on the general detection of plant diseases by using DCNNs has

demonstrated up to 99.97% accuracy [15], which has established benchmarks in the detection of agricultural disease. Studies based on CNNs, such as the ones by Kumar et al. [16] and Lu et al. [17], indicate that CNNs can be used to identify diseases in real-time.

The hybrid architectures aim at achieving as precise and efficient as possible. Another proposal was offered by Bedi and Gole [18], who suggested a hybrid model between CAE and CNN, which maintained the high performance of the model with a reduction in the cost of computing. Falaschetti et al. [19] applied a lightweight CNN to embedded hardware to use in mobile agriculture. Vision transformer-based models are emerging as strong rivals of CNNs. A combination of CNNs and ViTs was used to classify plant diseases by Thakur et al. [20], who showed an increase in accuracy and interpretability.

Further applications of agriculture are more complex such as applying the Mask R-CNN and YOLO-based models to locate Weligama Coconut Leaf Wilt at its initial stage, which is effective in the real world [21]. The ensemble learning and stacking have been used and as shown by Sharma et al. [22], the accuracy of the detection of the coconut leaf disease is 98.4%. Wijesinghe et al. [23] explored the case of detection systems involving multispectral imaging and SVMs UAVs. This demonstrated that remote sensing of coconut plantations can be done through other methods.

Model ResNet50 and similar ones remain successful in detecting various kinds of coconut leaf diseases. According to the findings of Kaur et al. [24], it was 98.7 percent accurate. The same was demonstrated by Gopalakrishna and Lingararajan [25], who revealed that coconut disease can also be predicted with the help of models that are based on MobileNetV2. Besides the research on coconut plantations, numerous papers on the application of CNNs and Faster R-CNN to detect rice leaf disease [26, 27] provide us with some rugged information and concepts to develop robust detection schemes. Studies based on transfer learning and ensemble fusion are characterized by high accuracy and generalization of different crops as well [28, 29, 30, 31].

These findings indicate that deep learning has gone a long way in identifying coconut leaf disease and plant pathology. However, it is possible to improve generalization, diversification of data, light models to deploy in real-time, and explainability to simplify its application to real life. A summary of literature review shown in table 2.1.

Table 2.1: Summary of Literature Reviewed.

Author (s)	Year	Title	Methodology	Key Findings
Subbaian et al. [1]	2024	Coconut leaf disease detection	YOLOv4	Achieved 88% F1-score and 85% mAP for leaf disease detection.
Singh et al. [2]	2021	Coconut disease and pest detection	CNN, MobileNet, InceptionResNetV2	Custom CNN achieved 96.94% accuracy.

Nivethitha et al. [5]	2022	Review on coconut disease detection	Deep Learning Review	Summarized CNN-based methods and challenges.
Muthulakshmi et al. [7]	2025	Coconut disease classification	DenseNet201, ResNet50, EfficientNetB0	Achieved up to 99.85% accuracy.
George et al. [8]	2023	Early detection of stem bleeding	Color thresholding	Achieved 80.36% pixel accuracy.
Thite et al. [4]	2023	Coconut tree disease dataset	Dataset creation	5,798 labeled images across 5 disease categories.
Wijethunga et al. [11]	2023	Coconut plant disease identification and management	Machine Learning & Image Processing	Developed an integrated system combining environmental monitoring & ML-based disease classification.
Santhi & Murugan [12]	2025	Cocoscan: Precision diagnostics for coconut leaf disease	InceptionV3, Web App, XAI	Real-time disease diagnosis with XAI-based visual explanations for user trust
Pandian et al. [15]	2022	Plant disease detection using deep CNN	14-layer custom DCNN	Achieved 99.97% accuracy with large augmented dataset across 58 disease classes.
Lu et al. [17]	2021	Review of CNN for plant disease classification	Survey/Review	Summarized state-of-the-art CNN architectures and highlighted research gaps.
Falaschetti et al. [19]	2022	Vision transformer + CNN hybrid for plant disease classification	ViT + CNN Hybrid	Improved accuracy and transparency using a lightweight ViT-CNN architecture.
Vidhanaarachchi et al. [21]	2025	Early diagnosis of coconut leaf wilt & caterpillar infestation	CNN, Mask R-CNN, YOLO	Achieved 90–97% disease accuracy and >95% caterpillar detection rate.

2.2.1 Similar Applications

Disease detection using the deep learning approach has been extensively used in agriculture, which has provided an insight applicable to the coconut plantations. Mohamed et al. created a mobile application based on Faster R-CNN which was able to locate pests in real-time with 98 percent accuracy. This shows that mobile apps can be applied in the field. CNN architectures were used by Li et al. to detect pests in rice and achieved a 82.4% accuracy. This indicates the significance of the existence of good extraction means in agriculture. SSD was also used by Liu et al. to locate pests in real-time with MobileNet, and they were correct 90.6 percent of the time. It demonstrates that lightweight models are applicable in cases where the number of resources is minimal. The accuracy of Nanni et al. [18] was 92.4% with the use of various CNN models, and Li et al. used YOLOv5-S with the accuracy of 89.6% to detect pests in real-time, which demonstrates the importance of the balance of speed and performance. These applications indicate that deep learning gains increasing significance in developing tools that can be useful and efficient in farming which can be utilized by a large number of individuals. Many of these tools directly affect the decisions to be made concerning the way to find coconut diseases.

2.3 Gap Analysis

The detection of coconut leaf disease is now much more successful, yet there are certain aspects that should be corrected shown in table 2.2. Many of the recent researches employ small data sets that lack the number of variations regarding lighting conditions, age, severity of disease, and the environmental conditions, complicating its application in different settings. The CNN-based models perform relatively well in controlled tests but they are not so good in the real world. It has few lightweight models that could be utilized on the mobile devices or at the edge, yet the plantations require on-site diagnosis. This is another big gap. A few studies consider more sophisticated architectures such as CapsNet or transformers, but few consider explainable AI techniques that may allow people to have more trust in AI, which is critical to make people adopt AI in the farming industry. Ensemble and hybrid approaches are improving but not a lot of experiments that investigate various types of coconut diseases as yet. In addition, little studies have been conducted on the integration of detection, classification, and severity estimation into a unified model. The proposed system addresses these gaps through training advanced deep learning models with the help of a structured five-class dataset, explainable AI (Grad-CAM), and placing much emphasis on real-time performance and model reliability.

Table 2.2: Gap Analysis

Features	Existing Coconut Systems	Proposed System
Multi-disease classification	No	Yes
Balanced dataset	No	Yes
Heavy + lightweight model comparison	No	Yes
Explainable AI (Grad-CAM)	No	Yes
Mobile/real-time feasibility	No	Yes
Early-stage detection	No	Yes
Comparison across multiple pretrained architectures	No	Yes
Scalability for large-scale deployment	No	Yes

2.4 Summary

This chapter provided a brief but important background on how to locate coconut diseases, and how deep-learning can be used to automate agricultural diagnostics. It has been demonstrated that CNNs, transfer learning, hybrid frameworks, and even transformers are effective at classifying plant diseases. Past studies show a solid performance in a variety of architectures but also reveal persistent difficulties, such as a lack of diversity of datasets, variation in performance under practical environments, and no integrated, transparent and lightweight systems to deploy to the field. The analysis of similar applications and gap analysis proves that the sphere is rapidly evolving, yet it is necessary to have accurate and easy to comprehend and efficient to apply models. This is what will form the methodology and the system that will be offered in the following chapters based on the insights gathered here.

Chapter 3

Research Methodology

In this chapter, the research methods applied in developing, testing, and developing the coconut disease detection system are explained. It outlines the methodological framework that was employed in the creation of the dataset, the model selection, preprocessing, training, Explainable AI integration, and evaluation strategies. It also provides reasons as to why to make disease classification more accurate and faster; some architectures, such as DenseNet121, EfficientNet-B4, and ensemble fusion models, were selected.

3.1 Methodology

The research used a systematic research process to come up with a reliable and scalable taxonomic system of the coconut leaf diseases. To begin with, much literature on the application of deep learning in the diagnosis of plant diseases was reviewed. It involved CNN architectures, mobile-deployable lightweight models, ensemble learning, and Explainable AI technologies such as Grad-CAM. The review allowed us to identify some of the key flaws of prior research, such as not having a balanced set of datasets, mistakenly grouped diseases, and models that are not user-friendly. The given methodology will aim to increase accuracy through fusion models, reduce computational requirements by using lightweight networks, balance the dataset, and provide the explainability to strengthen the trust in forecasting. Testing the performance, the comparison of various models, and making preparations to use it in real-time are also aspects of the methodology.

3.1.1 Overview

The paper discusses an engineered deep learning-based system that is trained to classify different coconut leaf diseases using a refined structure of preprocessing, augmentation, transfer learning, and model evaluation. The initial stage of the process will be to collect and prepare a collection of images of healthy and sick coconut leaves. The dataset is balanced and cleaned to ensure that the learning process is less biased. Preprocessing entails noise reduction, scaling of the samples, normalization of the samples, and inclusion of additional samples that are more varied to prevent overfitting. In the experiments, we examined several model architectures. Others were bulky, such as DenseNet121, which existed to assist with improved feature extraction, and others were not so bulky, such as EfficientNet-B4, which existed to allow us to run it on mobile devices or in resource-limited locations. Explainable AI is constructed by using Grad-CAM to provide disease-specific locales and assess the trustworthiness of the model. The entire pipeline ensures that the system has the ability to be utilized in real time, in areas with minimal resources, and in making feasible decisions in agriculture.

3.1.2 Proposed Methodology

The suggested methodology uses a multi-stage deep-learning model on the precise and computationally efficient classification of coconut disease shown in figure 3.1. It starts with dataset preprocessing, which consists of normalizing images, resizing, cleaning, and augmenting by rotations, changes in brightness, flips, scaling, shearing, and zoom changes. This augments strength and reduces imbalance of the data. The training phase includes trying out various architectures such as DenseNet121, MobileNetV2, MobileNetV3, ResNet50, and Xception. Complex visual features are obtained by using heavy models, whereas inference is fast by using lightweight models. Training It is done through transfer learning, fine-tuning, and regularization in order to speed up convergence. The measures of evaluation are accuracy, precision, recall, F1-score, and confusion matrices. Grad-CAM explainable AI is used after training to show the parts of the leaf that affect every prediction, which proves that the system makes medically significant choices. Figure 3.1 shows the entire process of the proposed methodology.

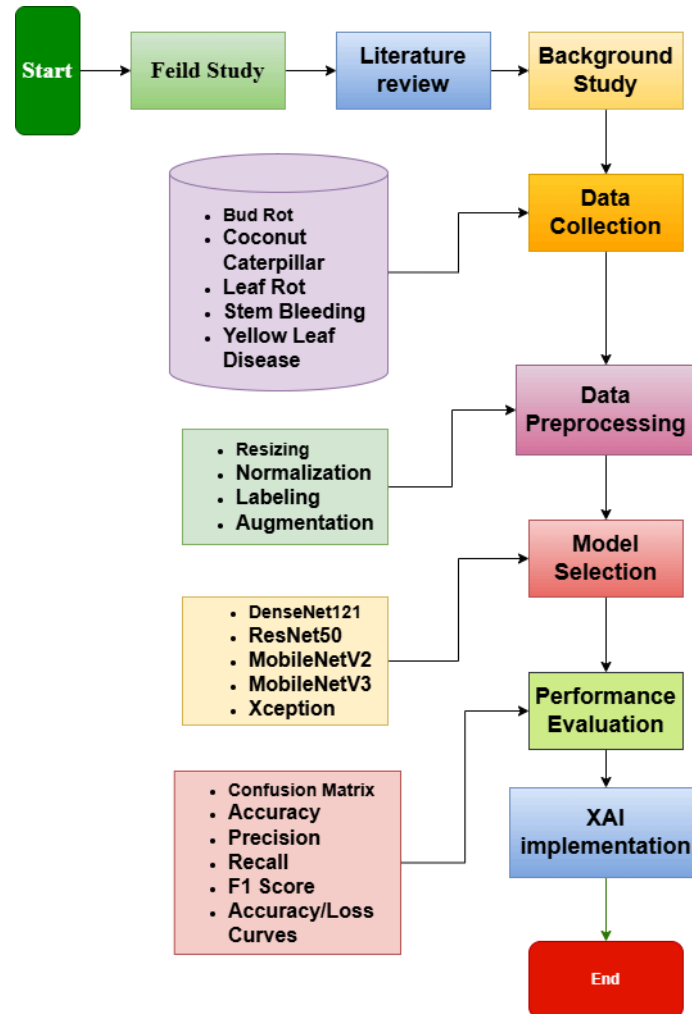


Figure 3.1: Methodology diagram

3.2 Detailed Methodology and Design

This section explains the entire procedure that was employed in the creation of the coconut tree disease detection system. It discusses how the dataset was collected, processed, cleaned, and augmented and divided into train and test parts to train the models. It also provides a summative conclusion of the constructed dataset that was utilized during the experiments. Various other alternatives were considered as the model was being developed. The first ones we looked at were more conventional machine learning classifiers, such as SVM and Random Forest, which we did not like since we did not find them well representing the visuals of diseased leaves. We also considered simple CNN architectures, which did not perform well in the situations when similarities within the same class were high. The last option was deep learning models such as DenseNet121, ResNet50, MobileNetV2, and MobileNetV3 due to the fact that they are highly effective at feature learning. We corrected the models to be more precise. Grad-CAM was introduced in order to simplify the system and thus make it more reliable and applicable in the real world of farming. This design ensures that accuracy, interpretability, and computational efficiency are all balanced.

3.2.1 Data Collection

The dataset used in this research was collected from multiple publicly available online collections and free image databases of coconut plant diseases. Since the raw data consisted of mixed classes and varying resolutions, and not all the samples were useful, all the images were inspected manually to ensure that they could be used in classifying coconut diseases. Following the cleaning process, this work adopted five predominant forms of coconut disease, which included Bud Rot, Coconut Caterpillar, Leaf Rot, Stem Bleeding and Yellow Leaf Disease. These classes demonstrate the most widespread and destructive issues with the coconut leaf that occur on the farms. Figure 3.2 shows an overview of the data set.



Figure 3.2: 5 images from each class.

3.2.2 Data Preprocessing and Cleaning

3.2.2.1 Data Cleaning

The data provided were duplicate images, extremely low-quality samples, and samples that were not relevant. These were eliminated using a mix of a manual check and automated validation checks. Images that did not provide a clear picture of one of the five categories of disease were discarded. This ensured that the information was highly precise and reduced the possibility of error in classification during training.

3.2.2.2 Data Resizing

The images were all rescaled to 224 x 224 pixels to ensure that the data was of the same size and fit into pretrained CNN models as an input. The resolution is a good compromise between computer efficiency and preserving the essential aspects of the disease, such as spots, textures, lesions, and color changes.

3.2.2.3 Normalization

To stabilize the numbers and accelerate the convergence of the model, the pixel values were scaled to a fixed range (typically 0-1 or the mean and standard deviation of ImageNet). Normalization ensures that there is never a difference in the contrast and lighting, which is beneficial in enabling models to learn patterns.

3.2.2.4 Dataset Balancing

To ensure that the model does not favor one dominant type of class, the dataset was balanced by ensuring that all five categories of diseases were represented equally. Oversampling and augmentation were only applied where necessary. This is a step that assists the model to handle the classes as ones that are of equal significance during the training process.

3.2.2.5 Data Augmentation

The dataset was augmented with a great variety of methods to ensure the model is more robust and the dataset is more varied. These included such elements as translation, rotation, shearing, scaling, flipping horizontally, and contrast alteration. It was aimed at altering the environment, which would occur in real life, such as changing the angle of the leaves, the lighting, and the camera angle. This step worked to the benefit of the dataset because it would add approximately 1000 images per class.

3.2.2.6 Training and Testing Split

After preprocessing and the addition of more data, the data was divided into 80% training and 20% testing. The parameters of the model were fine-tuned using the training set, and only in the end was the testing set used to determine how well it had worked. This division ensures that the assessment is correct and that there is no leakage of data.

3.2.2.8 Final Dataset Overview

The final curated dataset contains five balanced classes of Bud Rot, Coconut Caterpillar, Leaf Rot, Stem Bleeding, and Yellow Leaf Disease. Each class has 1000 augmented images and preprocessed images. The size of all images is equal to 224×224 pixels, and the values of pixels are equal. The data is clean, diverse, and well-structured, which is why it is an effective method to train a broad variety of different deep learning models and test their performance in the long term. Class distribution shown in the figure 3.3.

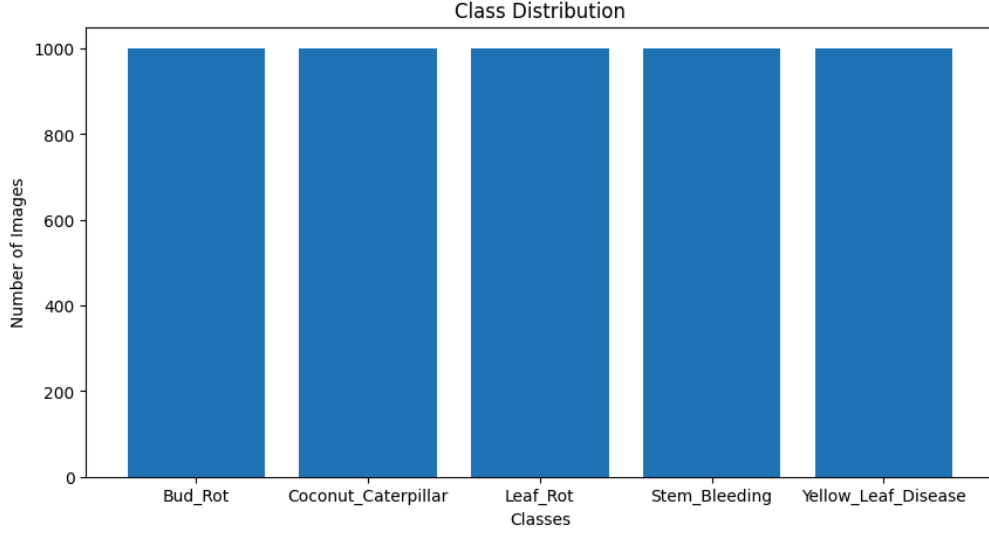


Figure 3.3: Class distribution of the dataset.

3.2.3 Model Architecture

3.2.3.1 Convolutional Neural Network (CNN)

One of the most essential deep learning models as far as image classification is concerned is the Convolutional Neural Network (CNN). The reason behind this is that CNNs are capable of acquiring spatial characteristics by independently using visual data. CNN's convolutional layers detect objects such as edges, textures, and shapes. The spatial dimensions are reduced by pooling layers, and the features that were found are sorted by fully connected layers. The simple arrangement typically consists of a Conv-Pool arrangement, which repeats itself multiple times, and then it is followed by dense layers. CNNs acquire hierarchical representations by stacking convolution and pooling operations. They start with simple, low-level structures and advance to more complex, high-level structures. Due to this quality, CNNs are efficient in the task involving pictures of plant leaves, where disease symptoms may be manifested in small color variations, or abnormal textures, or small lesions. In this research, CNNs would most suitably be the initial steps to the discovery of coconut disease since they are capable of learning features effectively and they are fast. A simple CNN architecture is shown in Figure 3.4.

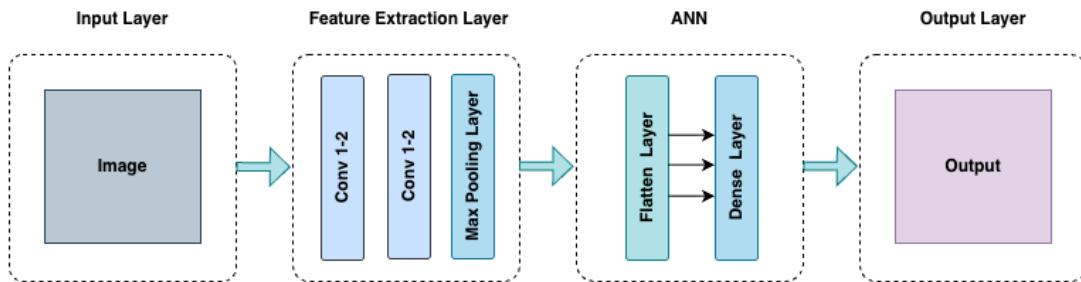


Figure 3.4: CNN Architecture

3.2.3.2 Transfer Learning

In this study, transfer learning will be used to solve the limitations of small size and variability of coconut leaf disease datasets. Pretrained models such as DenseNet121, ResNet50, MobileNetV2, MobileNetV3, and Xception are used as feature extractors in order to avoid overfitting and learning deep neural networks by default. These models were trained on the large-scale dataset ImageNet, and it means that they already have much information concerning colors, shapes, textures, and edges. The models soon get to know the patterns of coconut-specific diseases by holding their convolutional bases constant and modifying the final classification layers. This implies that they require smaller samples of training and take less time to calculate.

Such an approach has ensured faster convergence, better accuracy, and better generalization in different lighting scenarios and leaf images. It also allows the study to compare lightweight architectures to heavier ones, which contributes to the purpose of discovering effective and helpful solutions that could be implemented within the field. In general, transfer learning is highly significant in creating coconut disease detection models, which are effective in the real world. Figure 3.5 shows a generalized architecture of transfer learning [33].

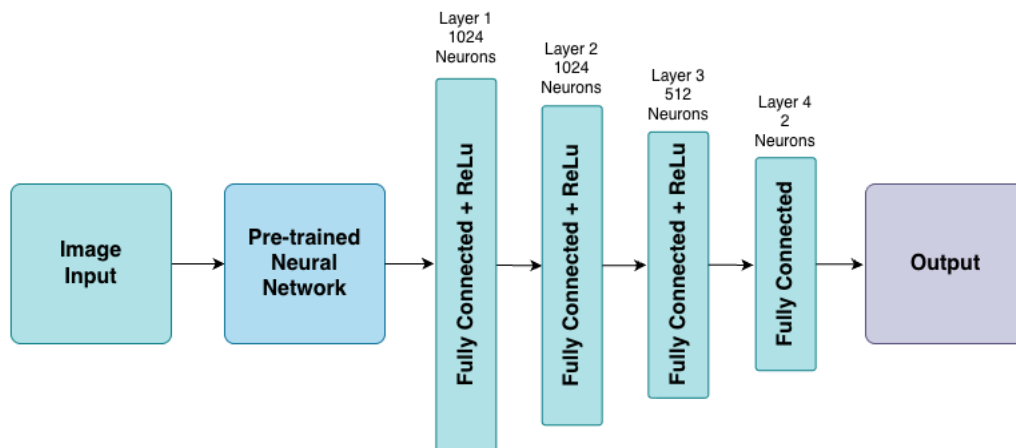


Figure 3.5: Transfer Learning Architecture.

3.2.3.3 Deep learning models

Deep learning is a form of machine learning that conditions artificial neural networks to discover patterns and associations within massive, complex data sets. These networks consist of layers of neurons that are interconnected. They both learn about extracting useful information out of raw data. Deep learning has taken significant forward leaps in image classification, object detection, natural language processing, and medical diagnostics since it is able to capture features that are not linear and that older models are unable to capture. Deep learning models in this study form the primary component of the system, which classifies coconut diseases. They aid the system in learning minor variations of disease symptoms over the image of leaves. This section discusses the

deep learning models, which were employed in the study and why they are effective when analyzing agricultural images with high accuracy.

3.2.3.3.1 DenseNet121

DenseNet121 is a convolutional net architecture, which combines dense skip connections between all the layers. This design allows the feature maps to travel smoothly across the network that accelerates gradient propagation and reduces the probability of the disappearance of the gradient during training. DenseNet121 promotes feature reuse, whereby earlier learned representations can be utilized by later layers. It is smaller in size and has fewer parameters than other deep architectures. DenseNet121 particularly performs well in tasks such as locating coconut disease since it is able to retain small information in leaf patterns, textures, and laceration forms. DenseNet121 is an effective backbone model in a variety of agricultural image classification tasks since it is efficient and able to generalize. A DenseNet121 architecture is shown in Figure 3.6 [34].

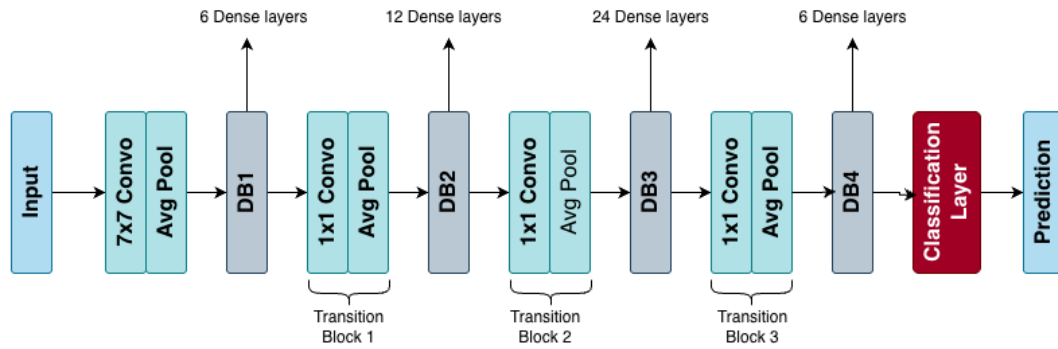


Figure 3.6: DenseNet121 Architecture.

3.2.3.3.2 ResNet50

ResNet50 is another extremely relevant deep learning model that is frequently employed to classify images. It also introduced residual connections that assist the network in addressing the degradation issue by allowing the gradient to pass through the shortcut paths. These block residual blocks allow you to train deeper networks without performance loss. ResNet50 is quite efficient in identifying low-level and high-level patterns in images using this. The architecture is well balanced in regard to its precision and cost, and as such it is a good option in the process of identifying diseases in plants where the model must identify the subtle change in the color, texture, and form of leaves. ResNet50 is selected to be one of the reference CNN models in this research since the model has been reported to perform well in plant disease classification, and it is also capable of processing images with diverse conditioning factors. In Figure 3.7, ResNet50 Architecture is shown [35].

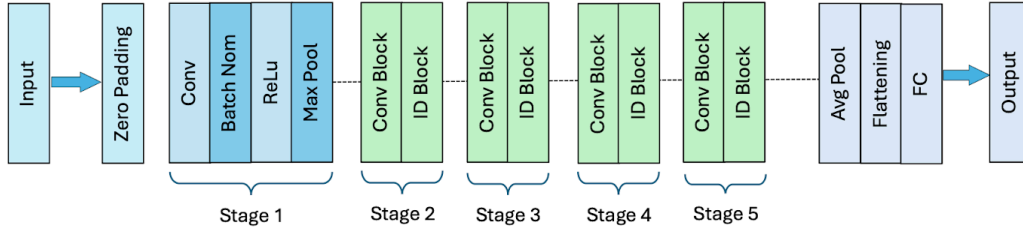


Figure 3.7: ResNet50 Architecture.

3.2.3.3.3 MobileNetV2

MobileNetV2 is a small neural network that is designed to be used in mobile applications and resource-scarce environments. It applies inverted residual blocks and depthwise separable convolutions to reduce the number of calculations but maintain a high level of accuracy. The linear layers reduce the changes that are unnecessary, and the bottleneck layers retain the significant features. MobileNetV2 is suitable to use in real-time farming applications since it is not limited to devices with high memory and processing capabilities such as smartphones and edge devices. MobileNetV2 is suitable to be deployed in the fields that could have limited computing resources due to a lack of power since the model can provide fast inference and low power usage to identify coconut disease. The architecture is shown in Figure 3.8 [36].

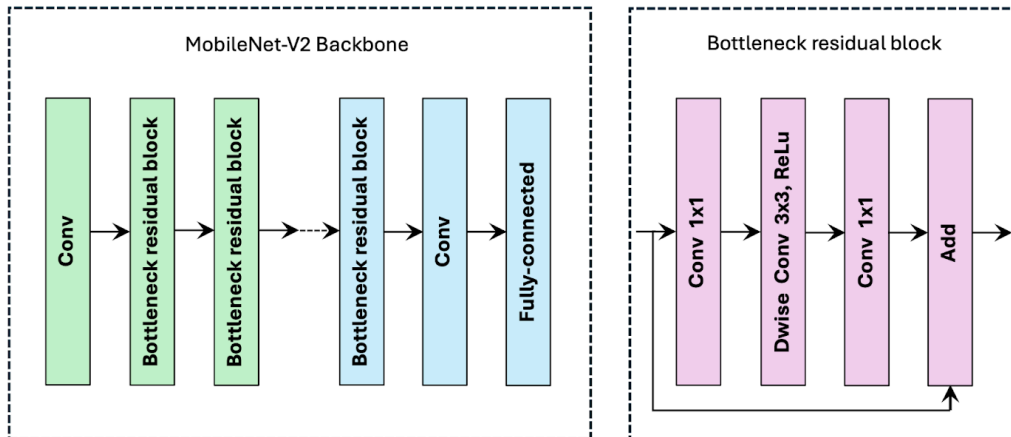


Figure 3.8: MobileNetV2 Architecture.

3.2.3.3.4 MobileNetV3

MobileNetV3 will utilize MobileNetV2, although it includes the neural architecture search (NAS) and squeeze-and-excitation (SE) modules to enhance the network automatically. It is available in two sizes, Small and Large. The two are designed to be

as efficient and precise as possible. MobileNetV3 is even more efficient in consuming computer resources than the former and performs better in a variety of benchmark datasets. This renders it ideal in the development of portable disease detection devices that can be used either on the mobile phones or embedded board. MobileNetV3 is suitable to use in real-time classification of coconut disease in a setting with scarce resources due to its lightweight design, SE attention and optimized convolutional block. Figure 3.9 shows the architecture's block diagram [36].

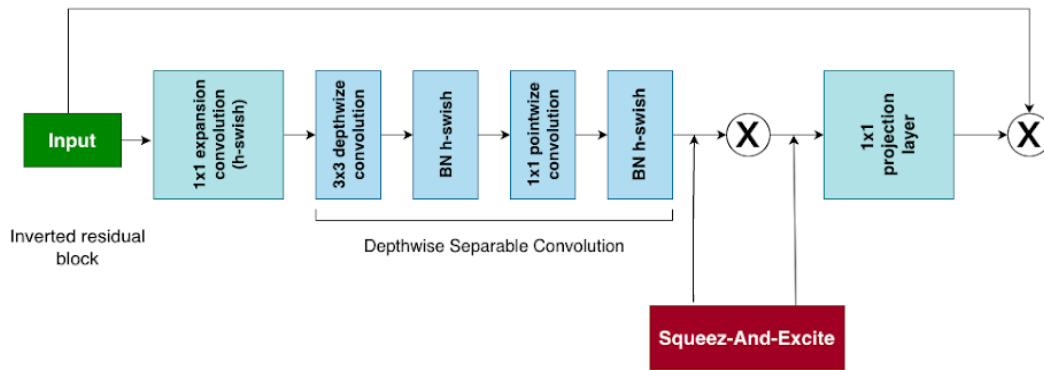


Figure 3.9: MobileNetV3 Architecture.

3.2.3.3.5 Xception

Xception is a CNN model that employs depthwise separable convolutions rather than depthwise regular convolutions. This is useful in making the model know more about depth and relationship of channels. It is based on the concept of Inception modules, but entirely distinctly separates the spatial and cross-channel feature extraction. This renders the architecture more potent and effective. Xception excels at complex visual tasks and it is called capable of detecting minor changes in texture and patterns. This is a valuable ability to distinguish between analogous symptoms of coconut disease. Since the model is competent and can be constructed in a relatively simple way, it is an appropriate place to start when comparing contemporary CNNs with transformer-based solutions to classifying agricultural images. The model architecture shown in Figure 3.10.

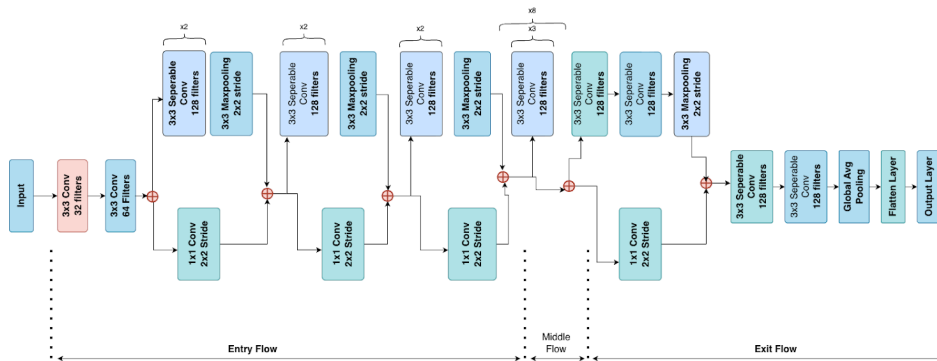


Figure 3.10: Xception Architecture.

3.2.4 Experiment

During this step, we would design and test various deep learning models to determine their performance on the curated coconut disease dataset. The paper has taken a look at the heavy and lightweight systems to understand the tradeoff between the accuracy, time of computation, and feasibility to deploy. In order to ensure that the comparison was fair, all models had been trained using the same preprocessed dataset and under similar experimental conditions. Fine-tuning pretrained weights, transfer learning, and measuring the performance of the model on a variety of different measurements were also a part of the experiment.

3.2.4.1 Heavy Model vs Lightweight Models

The experiments considered deep models such as DenseNet121, ResNet50 and Xception in addition to light models such as MobileNetV2 and MobileNetV3. The model with more parameters and extra layers of feature extraction normally allows heavy models to identify the fine disease-related patterns and complicated texture details. However, they require additional computing power, require more time to train, and are not as relevant to mobile or edge devices. Conversely, the lightweight models are more effective in field application since they may be applied in real time, they consume less memory, and they take a shorter time to make decisions. This comparison assisted us in identifying the optimal trade-off between performance and efficiency to practice farming in the real world.

3.2.4.2 Training and Fine-Tuning

The models were all trained using transfer learning with initially ImageNet weights being trained. Initial freezing of the base layers was to preserve the general feature representations, and the top layers were fine-tuned on the coconut disease dataset to acquire domain-specific patterns. We were able to discover the ideal values of the learning rate, batch size, and the number of epochs through experimentation. Fine-tuning assisted every architecture to optimize in the five disease categories that made it more precise and prevented it from overfitting. Data augmentation was employed to enhance generalization in the course of training, and early stopping was employed to conserve time and effort.

3.2.5 Model Evaluation

To have a complete picture of the model performance, we applied a variety of different measures to test the model performance. The measures of accuracy and loss indicated the extent to which the model was trained to work generally, whereas the measures of precision, recall, and F1-score indicated the extent to which the model predicted classes. Other tools like confusion matrices and line graphs were used trying to figure out the distribution of errors and training progress. A combination of these measures ensured that the models were thoroughly tested on their reliability, robustness and consistency.

3.2.5.1 Accuracy

Accuracy considered the proportion of images that were correctly detected in each of the classes. It provided a general overview of the performance of the model. The high accuracy indicated that the model was accurate in distinguishing the five types of coconut disease, whereas the low accuracy indicated that the model was either misclassifying or not learning well enough.

3.2.5.2 Loss

The measure of loss was the degree to which the model was erroneous in the training stage. It assisted in updating the tracking of the success of the use of gradient updates to minimize misclassification to the network. The curve, which decreased, indicated stable learning, whereas values of losses, which fluctuated or exceeded their threshold, indicated bad optimization or overfitting.

3.2.5.3 Precision

Precision was used to determine the ability of the model to detect a particular disease without any false positives. The precision of the model was very high such that it was certain and accurate in its predictions for each category of disease, a factor that made it effective in preventing wrongful diagnosis.

3.2.5.4 Recall

Recall made use of the ability of the model to detect all positive cases (real cases) in a disease class. The recall values were high, indicating that the model was able to detect even minor symptoms of illness and this reduced the risk of missing a diagnosis in the real world.

3.2.5.5 F1-Score

The F1-score calculated precision and recall as a single number, which provided a reasonable understanding of model prediction. It was particularly useful with datasets that were not balanced since it ensured that under-prediction and over-prediction were considered in the evaluation of a model.

3.2.5.6 Confusion Matrix

The confusion table presented true positives, false positives, false negatives, and true negatives of each category of disease. This provided us with much information concerning predictions of individual classes. It also assisted in discovering the most commonly misclassified diseases and provided recommendations on how to better preprocess or tune models.

3.2.5.7 Line Graph (Accuracy and Loss)

In order to view the change in learning with time, training and validation accuracy and loss were plotted as line graphs. These plots indicated whether the model was converging, whether it was overfitting the data, or whether it required additional tuning. They clarified to them the stability of the training and the general performance improvement.

3.2.6 Final Evaluation and Comparison

The final test after training all the models was to test their accuracy, speed, size, and speed of inference. Better models were typically heavy and consumed higher resources. Lightweight models, on the other hand, had equal performance but were very efficient. Through this comparison, it was possible to select a balanced architecture that would be effective in real-time systems to identify the coconut disease. The final findings were that lightweight models such as MobileNetV3 were effective even though they were quite precise in categorizing objects.

3.2.9 Explainable AI (Grad-CAM)

Explainable AI processes can make us learn how deep learning models arrive at their decisions. A form of visual explanation, Gradient-weighted Class Activation Mapping (Grad-CAM), is one of the most popular methods of visual explanation, displaying the elements of an image that have the most influence on the prediction of a model. Grad-CAM can also be useful in crop disease diagnosis, particularly since it allows the experts and end-users to know whether the model is examining actual disease symptoms and is not examining noise and other background factors. Incorporating Grad-CAM into this research aids in transparency, trust, and understanding enabling individuals to understand how this model functions. It also assists in the identification of model errors and the system is made more dependable when applied to real life farming scenarios.

3.3 Project Plan

This is an undertaking that is done in various organized stages. Every step identifies the key activities and objectives to be accomplished to establish the coconut leaf disease detection system.

Phase 1: Literature Review & Research Setup

- I. Examine existing methods of identifying coconut illnesses based on CNN, MobileNet, ResNet, DenseNet, Xception, and transformer models.
- II. Examine existing methods of identifying coconut illnesses based on CNN, MobileNet, ResNet, DenseNet, Xception, and transformer models.
- III. Based on similar agricultural disease detection systems, review their

- methodologies in order to determine suitable model structures in the project.
- IV. Bridging the gaps in the research and defining the way of the project.

Phase 2: Data Collection & Preprocessing

- I. Find images of the disease of the coconut leaves in several different sources.
- II. Dispose of low-quality, blurred, or irrelevant pictures.
- III. Select the last five classes: Leaf Rot, Coconut Caterpillar, Bud Rot, Stem Bleeding, and Yellow Leaf Disease.
- IV. Resize all the pictures to 224 x 224 pixels.
- V. Denormalize pixel values to ensure that no models receive different input.
- VI. In order to reduce bias in classes, balance the data.
- VII. Add to the image by use of translation, rotation, shearing, scaling, flipping, and contrast adjustment.
- VIII. Split the dataset into 20% tests and 80% training.

Phase 3: Model Development & Experimentation

- I. Transfer learning models such as DenseNet121, ResNet50, Xception, MobileNetV2, and MobileNetV3 can be used.
- II. Experiment with heavy and light architectures.
- III. Ensure that there is a uniform training of all models.
- IV. Enhance the learning rate, batch size, optimizer, dropout, and regularization.
- V. Monitor the performance of all five classes of diseases.

Phase 4: Model Optimization & Fine-Tuning

- I. Carry out much fine-tuning on the pre-trained layers.
- II. Early stopping Use early stopping to prevent overfitting.
- III. To add some generalization, dropout, and regularization.
- IV. Tune the hyperparameters in order to increase the accuracy of the model and reduce training time.
- V. Enabling lightweight models such as MobileNetV2 and MobileNetV3 to work in real time should be made easier.

Phase 5: Model Evaluation

- I. Evaluate the models using:
 - A. Accuracy
 - B. Loss
 - C. Precision
 - D. Recall
 - E. F1-score
- II. Calculate confusion matrices to be used in analyzing errors per class.

- III. Training/validation comparison Plots of accuracy and loss curve.
- IV. Compare the performance of the heavy and the lightweight model.
- V. Use Grad-CAM to represent areas of disease on the leaves.

Phase 6: Conclusion & Report Writing

- I. Compare the experimental outcomes and determine the best-performing model.
- II. Compare heavy/DenseNet/Xception and lightweight/MobileNet.
- III. Record the whole procedure, analysis, and results.
- IV. Write up the final research report and project documentation.

3.3.2 Research Resources

- I. Hardware**
 - A. Enabled environment with Gpus (Google Colab)
 - B. Enough RAM and storage to train several models.
- II. Software**
 - A. Python programming environment.
 - B. Tensorflow, Keras, scikit-learn, NumPy, Matplotlib.
 - C. GitHub for version control.
- III. Dataset**
 - A. Images of coconut leaf disease were taken in various online repositories.
 - B. After cleaning and selection, 5 disease classes.
- IV. Tools**
 - A. Google Drive for storage.
 - B. Colab training notebooks and visualization on the cloud.

3.3.3 Risk Management

- I. Data Scarcity**
 - A. Expand the variety of images with the help of augmentation techniques.
 - B. Use several sources to enhance the data-sets.
- II. Overfitting**
 - A. Adopt early termination and dropout.
 - B. Precisely check the performance of the monitors.
- III. Hardware Limitations**
 - A. Lightweight models can be used to optimize training.
 - B. Make good use of cloud GPU.

3.3.4 Expected Deliverables

- I. Coconut Leaf Disease Classification System:**

This disease belongs to the category of the Coconut Leaf Disease Classification System. Deep learning framework capable of detecting five typical coconut leaf diseases (Bud Rot, Coconut Caterpillar, Leaf Rot, Stem Bleeding, and Yellow Leaf Disease) with the aid of such models as DenseNet121, ResNet50, Xception, MobileNetV2, and MobileNetV3. This system aims at benefiting the farmers and workers in the agricultural sector by automatically detecting diseases through a picture of a leaf.
- II. Research Paper in Detail:**

An entire research article discussing the method of data collection, data cleaning, experimentation, model training, evaluation, gradient-cam, and model comparison. The paper demonstrates how lightweight and heavy CNN models may assist in locating coconut diseases and how transfer learning may assist in categorizing agricultural images.
- III. Well-Structured Codebase:**

An easy-to-clean and maintain repository with preprocessing, training, fine-tuning, evaluating, Grad-CAM visualization, and comparing model scripts. The codebase will not only facilitate future researchers expanding on the system, but it will also be possible to replicate results.
- IV. Model Performance Benchmarking:**

Accuracy, precision, recall, F1-score and confusion matrix were used to compare fully several architectures, including heavy (DenseNet121, Xception) and lightweight (MobileNetV2, MobileNetV3) ones. This is a comparison that should assist in deciding the best models to be used in agricultural applications that can be performed in real time or via mobile devices.
- V. Explainable AI Integration (Grad-CAM):**

Image maps with a visual demonstration of the areas of the coconut leaves that are infected. This makes it easier to understand and the users can know how the model makes decisions. This enhances the use of the system as it becomes more dependable in the diagnosis of issues in the field.
- VI. Practical Agricultural Impact:**

A disease detection workflow, which can be transformed, to both run on mobile devices or in real time. The system assists in preventing the loss of crops, it becomes less important to use expert inspection and helps in supporting intelligent agriculture initiatives that are currently trying to address the coconut disease by automating the process of detecting the disease at its initial stages

3.4 Task Allocation

This table represents the schedule of the key activities during every phase of the project, from week 12 to week 48.

Table 3.1: Task Allocation

Tasks	Weeks																			
	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48	
Data collection phase																				
Preprocess all the data																				
Model training																				
Create an application.																				

3.5 Summary

This chapter gives an in-depth description of the methodology used as a research to develop the classification system of coconut leaf disease. The chapter describes the concept of transfer learning to classify 5 large coconut diseases using some of the most popular deep learning models, including DenseNet121, ResNet50, Xception, MobileNetV2, and MobileNetV3. All the models were trained using a well-selected and pre-processed dataset. The images were washed, scaled to 224x224, normalized, balanced and augmented by translation, rotation, shearing, scaling, flipping and contrast alteration. The methodology also demonstrated the manner in which performance and computational efficiency can be compared by the use of both heavy and lightweight architecture. This assisted in the generation of a system that can be real time and mobile phone usable. The entire process of the experiment was also reviewed in the chapter including the division of the training and the testing data, fine-tuning, testing the model, and comparing its performance in terms of accuracy, precision, recall, F1-score, confusion matrix, and loss/accuracy line graphs. To simplify it, the Explainable AI techniques such as Grad-CAM were applied to display the parts of the body influenced by the disease and ensure the clarity of predictions. The chapter also discussed the general plan of the project, the resources that the project would require, management of risks and the deliverable that was to be expected. Such an approach establishes a complete deep learning-based coconut disease detection system, which is scalable, reliable, and useful. It can assist in the preliminary diagnosis and enrich contemporary agricultural practices.

Chapter 4

Implementation and Results

This chapter explains the implementation of the proposed system of classifying coconut diseases with the set-up environment, dataset, model training and verification of the results. It also presents the findings of the testing that was carried out on some of the deep learning models such as DenseNet121, ResNet50, Xception, MobileNetV2 and MobileNetV3. Subsequently, it discusses their performance in key measures.

4.1 Environment Setup

4.1.1 Hardware Requirements

The hardware configuration is highly significant in dealing with the training load of deep learning models, as far as image-based classification is concerned. The computationally expensive deep architecture such as DenseNet121 and Xception required far less training time, and the complex arithmetic it required was also performed by a system with a powerful GPU. Although attempting to increase the size of the batch or the depth of the model, the use of the GPU can ensure that model training, backpropagation, and parallel processing are well supported. In addition to the support of the GPU, there was also a requirement of sufficient RAM and processing power to support data preprocessing and augmentation. The storage space was required to be at least 100 GB to store the raw dataset, raw images that had been preprocessed, training logs, checkpoints, and visual outputs of experiments.

4.1.2 Software Requirements

The Python language was used to establish the software environment due to its numerous machine learning and deep learning-related tools. This was implemented in Windows. Installation of CUDA and cuDNN ensured that the TensorFlow and other structures could utilize the gpu resources in the most optimal manner. Python was predominantly used to create deep learning pipelines, process data, train models, and create visualizations and verify performance. This is a consistent software configuration that ensured that all was well at all project phases.

4.1.3 Frameworks and Libraries

The project relied extensively on the recent deep learning models that facilitate the construction of models. The primary libraries that were used to develop and train models were TensorFlow and Keras due to their flexibility and an in-built support of the ready-made architectures, such as DenseNet121, ResNet50, Xception, MobileNetV2, and MobileNetV3. Transfer learning has the best tools in these frameworks and hence it is easy to use big pre-trained networks to classify coconut diseases. Our other Python packages, such as

NumPy and Pandas, helped us to manipulate and process the data, whereas Matplotlib and Seaborn were used to create training curves and model performance plots. Scikit-learn played a significant role in the evaluation process as it assisted in such activities as classification report generation, confusion matrix generation, and dataset division.

4.1.4 Tools of Dataset Handling and Preprocessing.

The data used to conduct this project consisted of images that have been taken in various online repositories. It concentrated on five predominant types of coconut diseases, including the Bud Rot, Coconut Caterpillar, Leaf Rot, Stem Bleeding, and Yellow Leaf Disease. All the pictures were preprocessed before training the model using the tools such as PIL and OpenCV to ensure that all the images were of the same size (224x224), equal pixel values, and that all the pictures were in a similar format. Keras ImageDataGenerator and the Albumentations library were used by us to artificially add variety to the dataset and assist the models in understanding powerful patterns. By applying augmentation techniques, such as rotation, translation, scaling, shearing, flipping, and contrast manipulations, it was possible to ensure that the models did not get stuck on overfitting and prepare to transform in the real world. Granting of label encoding and balancing of the dataset to make sure that all classes were equally represented also belonged to this phase.

4.1.5 Version Control and Collaboration

The various versions of the code were monitored through Git, which assisted in keeping the flow of work clean and in order. Commits also recorded all significant changes to the codebase, including preprocessing modifications and modifications to model parameters. The remote repository was GitHub, and this was easily accessed, backed up, and collaborated with. Posting the project on GitHub allowed us to see the picture more clearly, track the history of the experiment, and compare and describe the models.

4.1.6 Model Training Configuration

The models were trained with the help of transfer learning. This entailed training the pre-trained architectures such as DenseNet121, ResNet50, Xception, MobileNetV2, and MobileNetV3 on the coconut disease dataset. To initialize these models, we used ImageNet weights so that they would use already learned features that were helpful in image classification. A batch size of 32 was used to train in order to achieve a stable convergence. Categorical cross-entropy was the loss function we used since the given job was to classify various classes. An 80-20 training/testing split was employed to ensure that the distribution of all five classes of diseases was balanced. The training configuration also included such a connection as callbacks, such as early stopping and model checkpointing, to prevent overfitting the model and save the most successful one.

4.1.7 Evaluation Setup

In order to be able to determine the performance of the trained models, we needed to compute a variety of significant performance values that indicate the extent to which they were able to classify data. The accuracy, precision, recall, and F1-score were determined

using scikit-learn tools. These measures indicated the extent to which each model could differentiate between the five different types of coconut diseases. Confusion matrices were created to determine the performance of each of the classes and potential misclassifications. The accuracy and loss of training and validation line graphs made it simple to observe the manner in which the model was converging and the stability of the training. Such an evaluation arrangement allowed conducting a comprehensive and trustworthy comparison of various deep learning structures.

4.1.8 Tools of Working and Communication.

The primary locations of writing code and documenting procedures, visualizing the results, and performing iterative testing were Google Colab and the development and testing process. The preprocessing pipeline could be enhanced more easily, the hyperparameters could be changed, and the model output could be studied in its interactive environment. Some of the tools proved to be helpful in terms of communication and coordination, such as Google Meet, Zoom, and Telegram. They simplified the process of talking about things, sharing updates, and maintaining schedules of the experiments. These tools provided the ease of people working together, and assisted in maintaining the order of the research process.

4.2 Testing and Evaluation/Performance/Comparative Analysis

In the testing part, we trained and tested five deep learning models, namely DenseNet121, ResNet50, MobileNetV2, MobileNetV3 and Xception, on the coconut dataset. This dataset consisted of five classes, namely the Bud Rot, Coconut Caterpillar, Leaf Rot, Stem Bleeding and Yellow Leaf Disease. All the models were trained equally, using the same preprocessing and the same augmentation strategies and an 80-20 split as between training and testing. We measured performance using accuracy, precision, recall, F1-score, and confusion matrices and loss-accuracy learning curves. The performance of all models was good with regard to classification, however, when it comes to generalization, stability, and computational efficiency, they differ. As it was anticipated, the scores of heavier architectures such as DenseNet121 and ResNet50 were nearly perfect. Conversely, mobile models such as MobileNetV2 and MobileNetV3 were also good but were much cheaper to execute. MobileNet V2 was the most suitable based on the results of the experiment since it was the most balanced in terms of accuracy, efficiency, and robustness with regards to real-time diagnosis and explainability using Grad-CAM.

4.2.1 Training and Validation Curve Analysis (Accuracy & Loss)

4.2.1.1 MobileNetV2

The performance of MobileNetV2 was extremely stable, and the training accuracy was 99.8 and the validation accuracy was 98.7. The training loss reduced to 0.1 and the validation loss remained at 0.4 which is an indicator of high generalization. The curves were not moving up and down wholly, which indicated that MobileNetV2 is potent despite being light. Figure 4.1 shows the curves for accuracy and loss of the model.

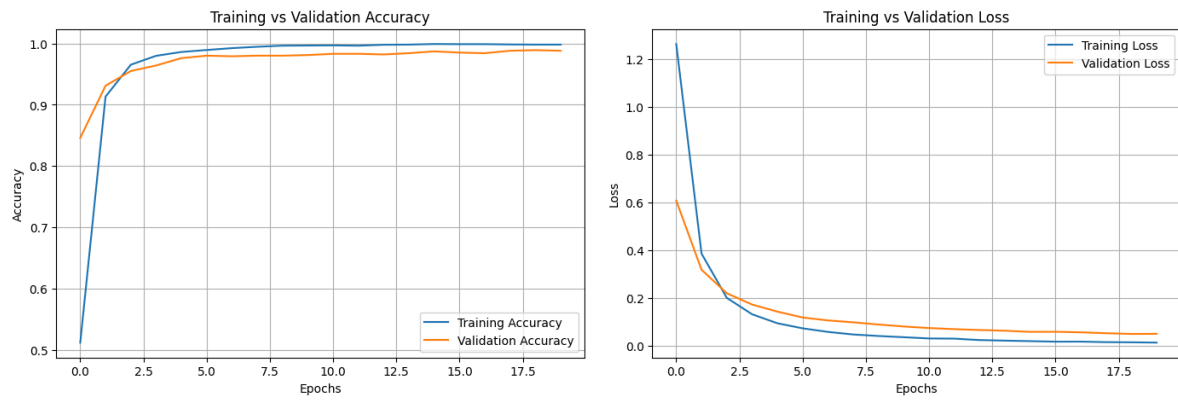


Figure 4.1: MobileNetV2 Accuracy and Loss

4.2.1.2 MobileNetV3

MobileNetV3 also did not take much time to develop and its training accuracy was 99.82 and validation accuracy was 97.60. The training loss decreased to 0.1% and the validation loss also increased to 0.6%. The curves were relatively stable, although the decentralization between training and validation loss was higher than in the case of MobileNetV2, indicating that performance on generalization was somewhat lower. The accuracy and loss curves are shown in Figure 4.2.

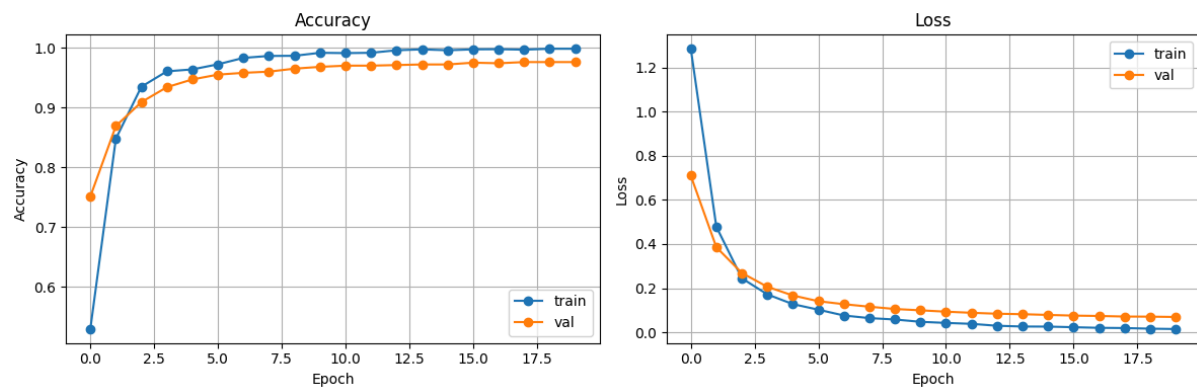


Figure 4.2: MobileNetV3 Accuracy and Loss

4.2.1.3 DenseNet121

Figure 4.3 shows that the training and validation accuracy curve of DenseNet121 increased rapidly and converged to 100% and 99.99% meaning that the model was able to learn very well at the beginning. The two losses were very close to zero, and this demonstrated that the model discovered patterns of diseases that were not similar to one another but they were not overfitting. The reason behind the minor difference between the training curve and the validation curve indicates that the model has a high generalization, which is consistent with how DenseNet121 links things.

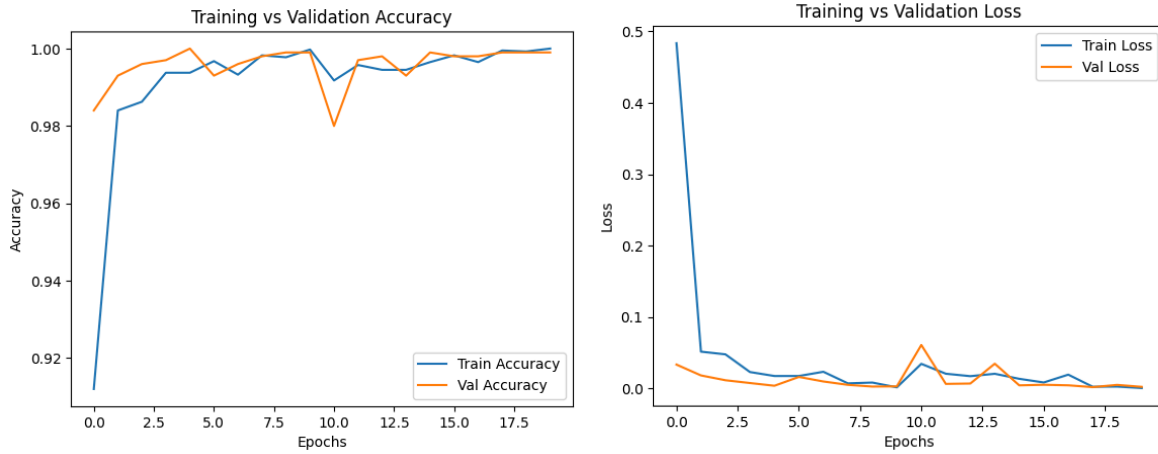


Figure 4.3: DenseNet121 Accuracy and Loss

4.2.1.4 ResNet50

In Figure 4.4 we can see that, ResNet50 also performed similarly with the training accuracy of 100 percent and the validation accuracy of 99.90 percent. The loss curves continued to decrease and became very close and that was indicative of residual connections being effective to prevent gradient degradation. The training and validation curves shown in Figure 4.4 shows that curves were not divergent, and therefore the model converged easily.

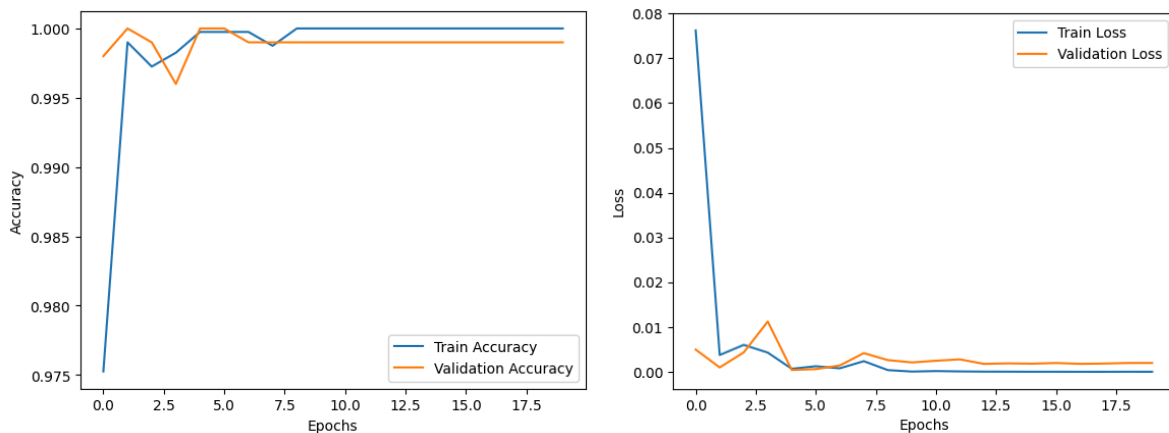


Figure 4.4: ResNet50 Accuracy and Loss

4.2.1.5 Xception

The two Xception curves continued to improve, and training accuracy was 98.22 percent, and validation accuracy was 98.69 percent. The curves of fuzzy loss were smooth, and the training loss was 0.07%, and the validation loss was 0.08%, and this indicated that Xception generalized the loss well. Still, more computing power was required to reach this stage of the model. Curves are shown in Figure 4.5.

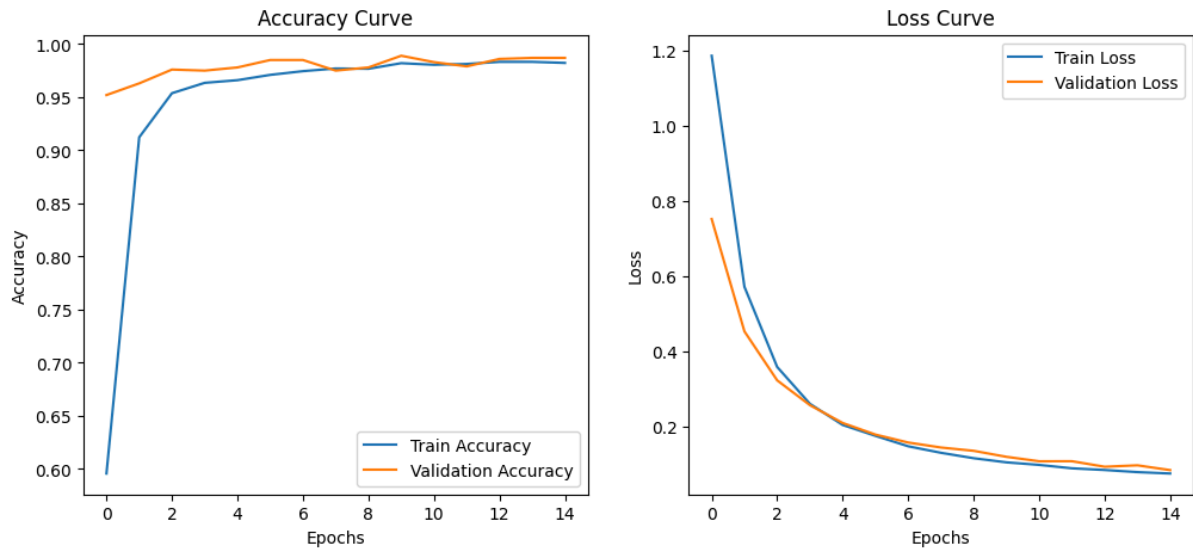


Figure 4.5: Xception Accuracy and Loss

4.2.2 Evaluation Metrics Analysis (Precision, Recall, F1-Score)

4.2.2.1 MobileNetV2

The results of MobileNetV2 were nearly ideal: Precision = 0.99, Recall = 0.99, and F1-Score = 0.99. These findings indicate that the model is effective in each of the classes and can be applied in a large variety of situations. The model was as effective as heavy architectures, and also it was not expensive to operate. Figure 4.1 shows the full classification report of MobileNetV2.

Table 4.1: Classification Report of MobileNetV2

Class	Precision	Recall	F1-Score	Support
Bud_Rot	0.96	1.00	0.98	200
Coconut_Caterpillar	1.00	1.00	1.00	200
Leaf_Rot	1.00	0.94	0.97	200
Stem_Bleeding	1.00	1.00	1.00	200
Yellow_Leaf_Disease	0.99	1.00	0.99	200
Accuracy			0.99	1000
Macro avg	0.99	0.99	0.99	1000
Weighted avg	0.99	0.99	0.99	1000

4.2.2.2 MobileNetV3

In table 4.2 we can see MobileNetV3 is also showing good results, albeit not as good as MobileNetV2. It had the Precision, Recall, and F1-Score values of 0.98. The minor decrease is attributed to an overlapping nature of patterns of certain types of diseases that are thus classified wrongly.

Table 4.2: Classification Report of MobileNetV3

Class	Precision	Recall	F1-Score	Support
Bud_Rot	1.00	1.00	1.00	200
Coconut_Caterpillar	0.96	1.00	0.98	200
Leaf_Rot	0.99	0.93	0.96	200
Stem_Bleeding	1.00	0.95	0.97	200
Yellow_Leaf_Disease	0.93	1.00	0.97	200
Accuracy			0.98	1000
Macro avg	0.98	0.98	0.98	1000
Weighted avg	0.98	0.98	0.98	1000

4.2.2.3 DenseNet121

DenseNet121 had the highest score on each of the tests: Precision = 1.00, Recall = 1.00, and F1-Score = 1.00. This implies that the model is capable of categorizing all five disease classes in a perfect fit, which shows how well the feature could be extracted as well as how stable it is. Full details shown in Figure 4.3.

Table 4.3: Classification Report of DenseNet121

Class	Precision	Recall	F1-Score	Support
Bud_Rot	0.99	1.00	1.00	189
Coconut_Caterpillar	1.00	1.00	1.00	191
Leaf_Rot	1.00	1.00	1.00	213
Stem_Bleeding	1.00	1.00	1.00	204
Yellow_Leaf_Disease	1.00	1.00	1.00	203
Accuracy	1.00	1.00	1.00	1000
Macro avg	1.00	1.00	1.00	1000
Weighted avg	1.00	1.00	1.00	1000

4.2.2.4 ResNet50

We can see in Figure 4.4 the results of ResNet50 were the same as DenseNet121, Precision = 1.00, Recall = 1.00, and F1-Score = 1.00. This demonstrates that ResNet50 successfully detected minute variations between diseases and was able to classify all samples of the test with the correct classification.

Table 4.4: Classification Report of ResNet50

Class	Precision	Recall	F1-Score	Support
Bud_Rot	1.00	1.00	1.00	189
Coconut_Caterpillar	1.00	0.99	1.00	191
Leaf_Rot	1.00	1.00	1.00	213
Stem_Bleeding	1.00	1.00	1.00	204
Yellow_Leaf_Disease	1.00	1.00	1.00	203
Accuracy	1.00	1.00	1.00	1000
Macro avg	1.00	1.00	1.00	1000
Weighted avg	1.00	1.00	1.00	1000

4.2.2.5 Xception

Xception scored highly on the evaluations, as expected of its accuracy during validation. Its accuracy to validate was 98.69 and its precision, recall and F1-score indicated that it was able to locate small disease features. But it was not better than DenseNet121 or ResNet50.

Table 4.4: Classification Report of ResNet50

Class	Precision	Recall	F1-Score	Support
Bud_Rot	1.00	1.00	1.00	189
Coconut_Caterpillar	1.00	0.99	1.00	191
Leaf_Rot	1.00	1.00	1.00	213
Stem_Bleeding	1.00	1.00	1.00	204
Yellow_Leaf_Disease	1.00	1.00	1.00	203
Accuracy	1.00	1.00	1.00	1000
Macro avg	1.00	1.00	1.00	1000
Weighted avg	1.00	1.00	1.00	1000

4.2.3 Confusion Matrix Analysis

4.2.3.1 MobileNetV2

MobileNetV2 had the cleanest confusion matrix among all the lightweight models, and the number of samples that were wrongly classified was few. All classes had true positives that were always high. The minimal errors that were discovered were minor and did not impact the overall performance significantly. This high consistency resulted in MobileNetV2 being an even lighter model. Confusion Matrix results shown in Figure 4.6.

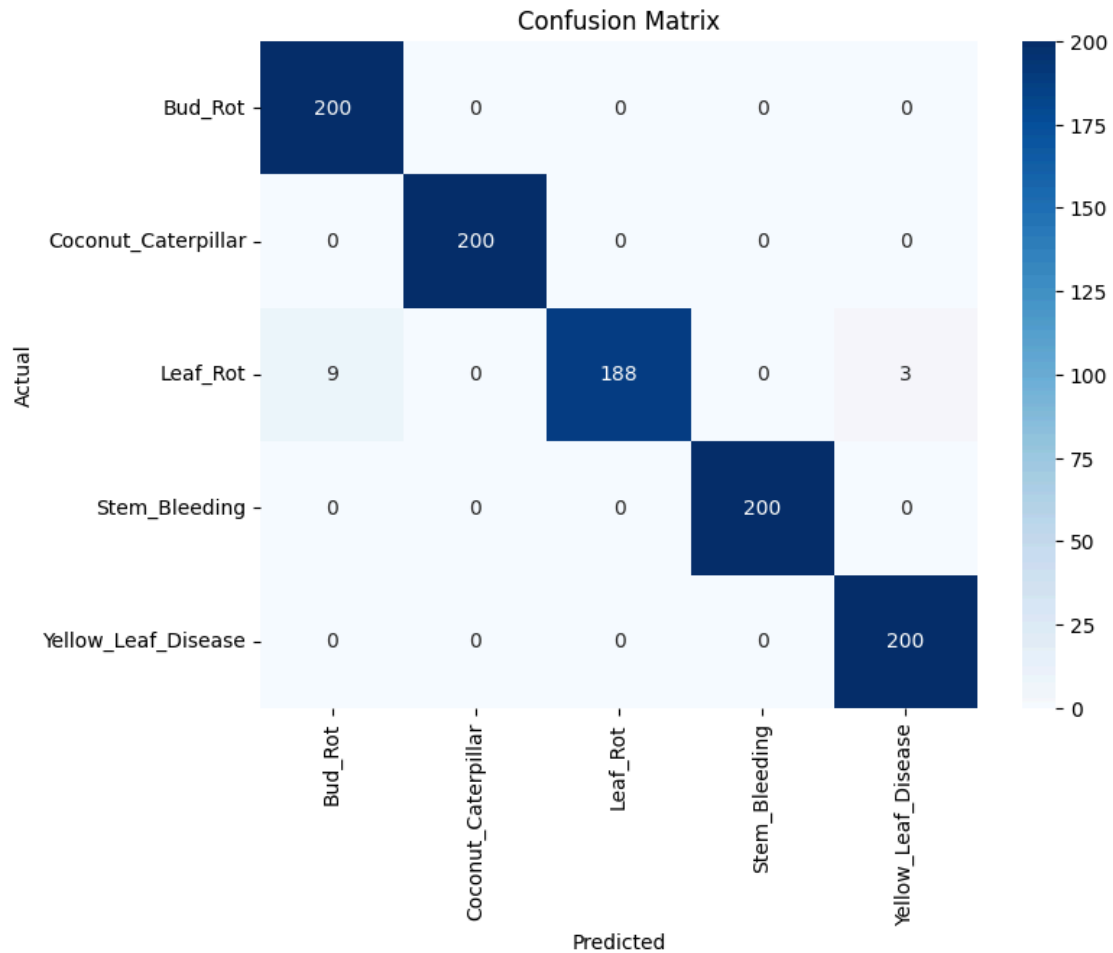


Figure 4.6: Confusion Matrix MobileNetV2

4.2.3.2 MobileNetV3

Figure 4.7 shows that MobileNetV3 had additional misclassifications than MobileNetV2. The majority of the errors occurred in between the similar-appearing diseases, such as the Leaf Rot and Yellow Leaf Disease. Overall accuracy of the model was still high, though.

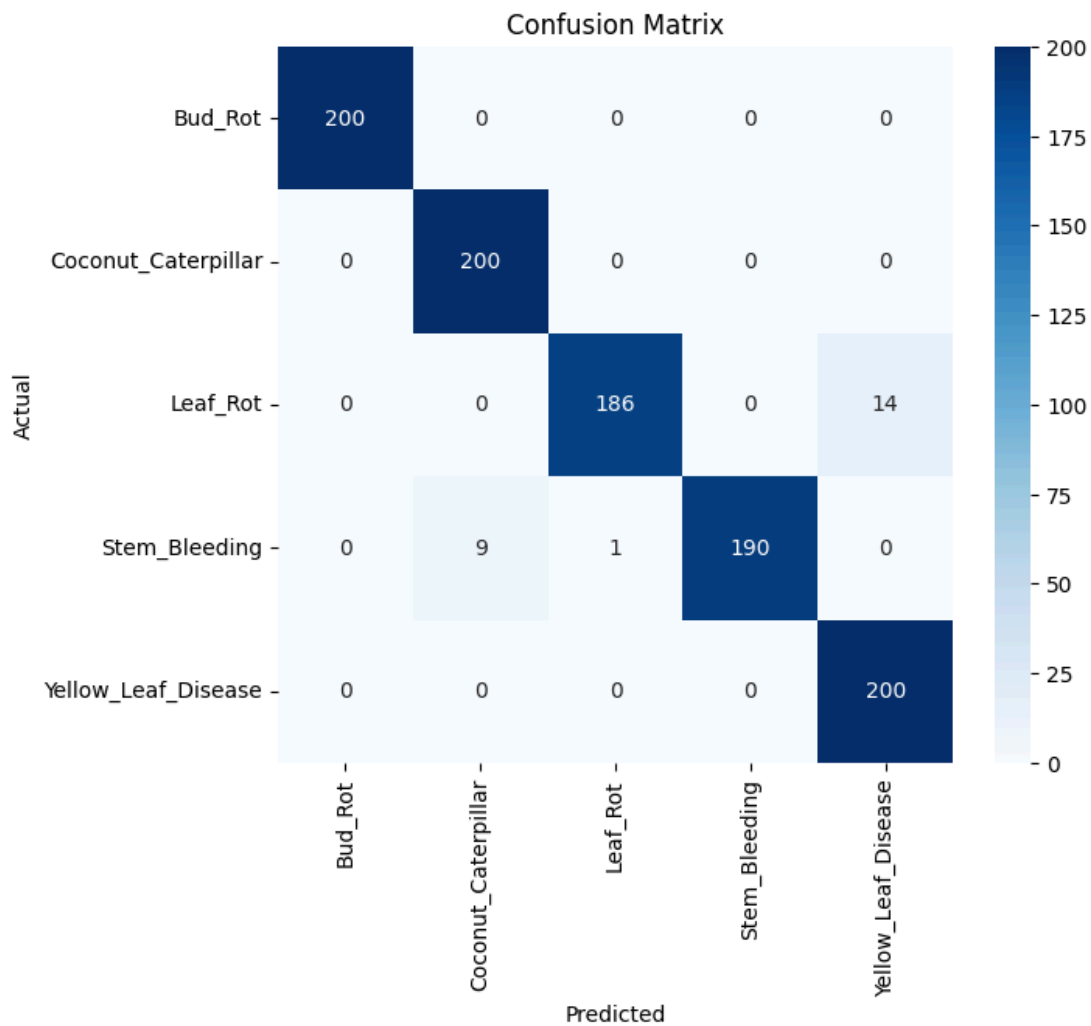


Figure 4.7: Confusion Matrix MobileNetV3.

4.2.3.3 DenseNet121

In Figure 4.8 DenseNet121's confusion matrix indicated that every five-class was accurately classified, and all the samples have been labeled correctly. The highest values were the diagonal ones and only 1 wrong classification was registered and this is to show that the features were quite good in making the distinction between them.

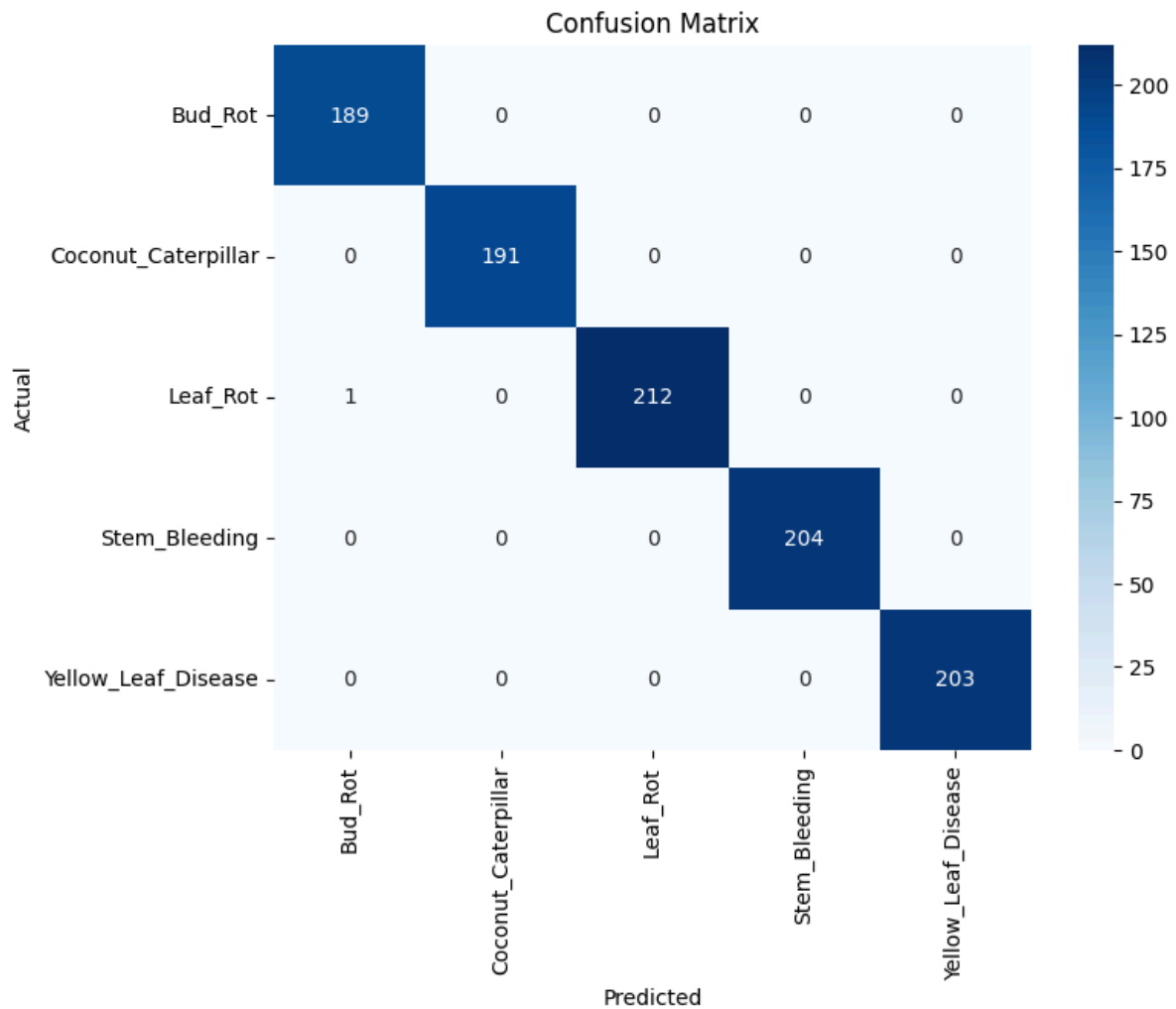


Figure 4.8: Confusion Matrix DenseNet121

4.2.3.4 ResNet50

DenseNet121 was also perfect in making predictions similar to ResNet50 shown in Figure 4.9. All the classes scored 100 percent accurate except 1 misclassification between Coconut Caterpillar and Stem Bleeding, indicating that the model can be effective in retrieving structural features.

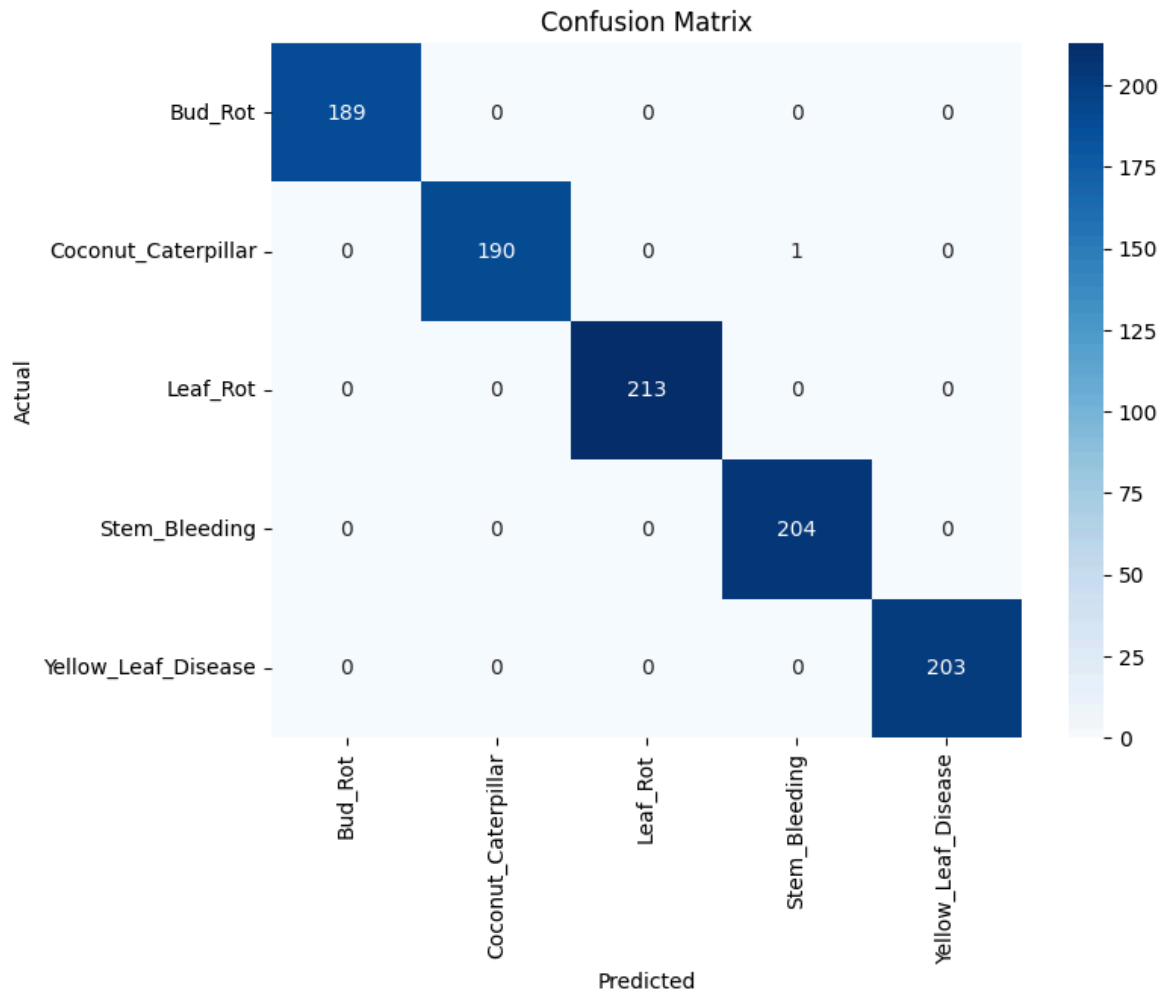


Figure 4.9: Confusion Matrix ResNet50.

4.2.3.5 Xception

The confusion matrix of Xception shown in Figure 4.10, indicated that the model was very effective in classifying items, although with more mistakes than models of MobileNet. This indicates that Xception had good qualities to learn complex features but in certain instances, its internal complexities ensured that it was overfitting.

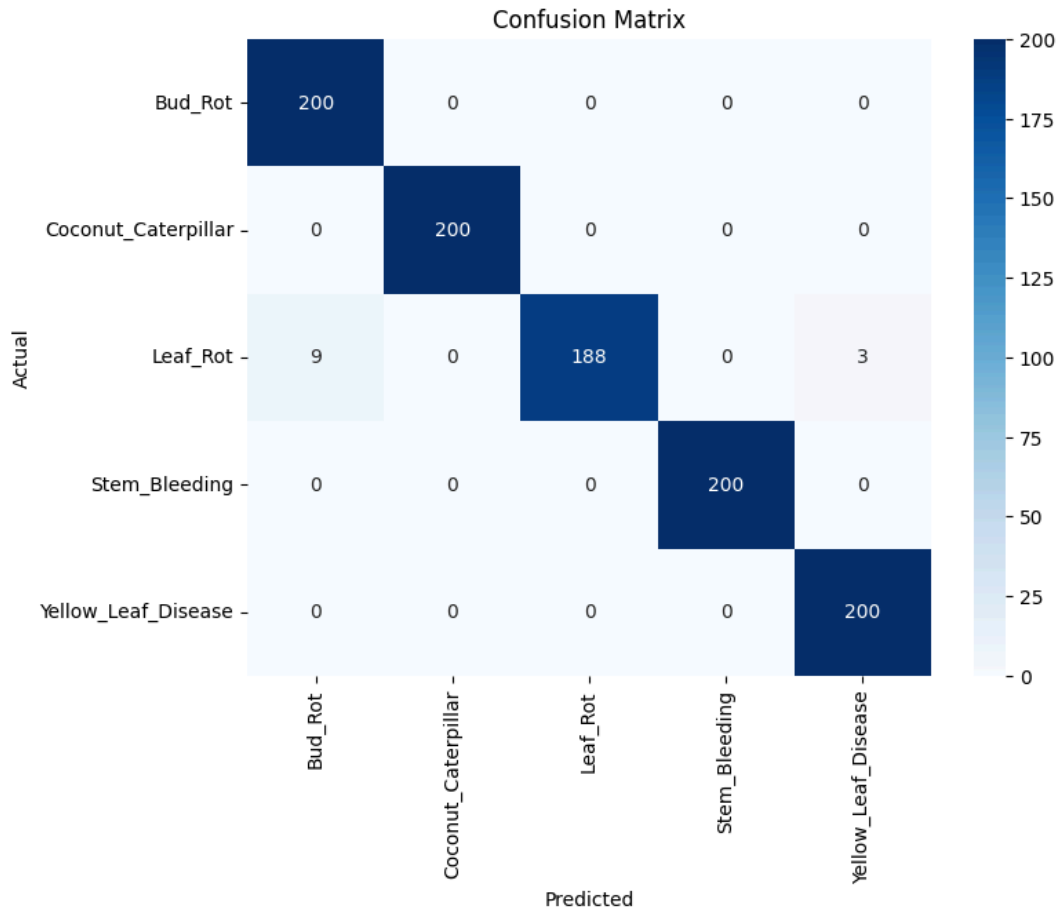


Figure 4.10: Confusion Matrix Xception.

4.2.4 Comparative Analysis between All Models

DenseNet121 and ResNet50 were the best models in terms of raw performance as they all had an ideal evaluation metric. However, such models are very strict with the computers, require more time to train and cannot be applied in mobile or real-time applications. Xception also performed excellently, however, it required a significant amount of computing power. MobileNetV2, in contrast, was very well-balanced in terms of accuracy (98.79% validation accuracy) and complexity and scored well in terms of the metric (0.99 Precision, Recall, and F1-Score). The confusion matrix of the model indicated the number of errors was very low and the learning curves were constant and the model converged within a short period. MobileNetV3 was also performing well yet it failed to generalize as MobileNetV2. We selected the best model, MobileNetV2 due to the following reasons: it was the most efficient, accurate, and the model could be easily used in the field since it was stable in generalization. Then we applied Grad-CAM explainability on it to demonstrate disease-specific activation patterns to achieve transparency and credibility.

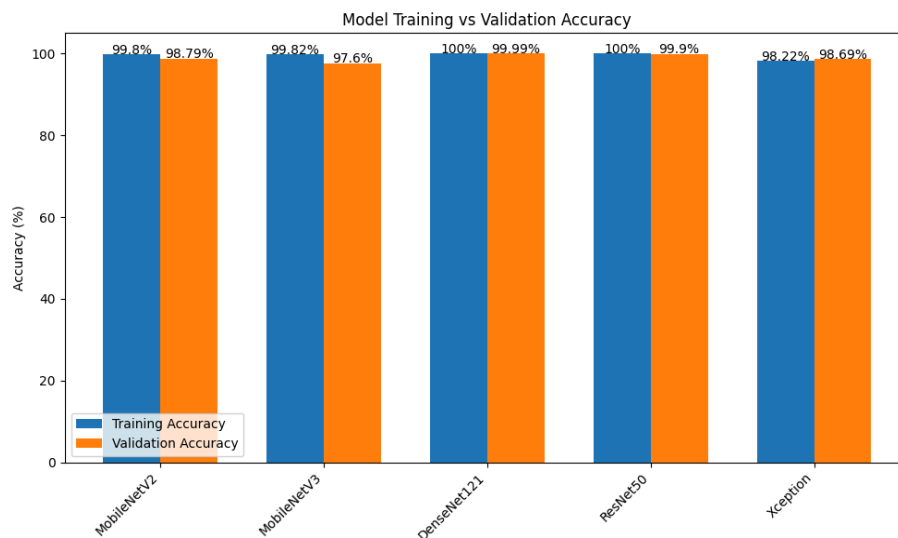


Figure 4.11: Comparison of accuracies between all models.

Figure 4.11 is a bar chart that has made a comparison between training and validation accuracies of the five deep-learning architectures namely: MobileNetV2, MobileNetV3, DenseNet121, ResNet50, and Xception. The two models that performed the best were DenseNet121 and ResNet50, as they both had a training accuracy of 100 and validation accuracies of 99.99 and 99.90 respectively. This demonstrates that they are very generalizing. MobileNetV2 performed also not bad, having a training accuracy of 99.80 and a validation one of 98.70. MobileNetV3 was even more successful, achieving a training accuracy of 99.82% and a validation accuracy of 97.60. This demonstrates that the two models are balanced both on performance and cost. Xception performed somewhat worse than the other models, yet it also was a good performer, with a training accuracy of 98.22 and validation accuracy of 98.69. The most accurate models in general are the denseNet 121 and ResNet50. Variants of MobileNet were also computationally efficient and had high accuracy. Xception was also slightly less accurate.

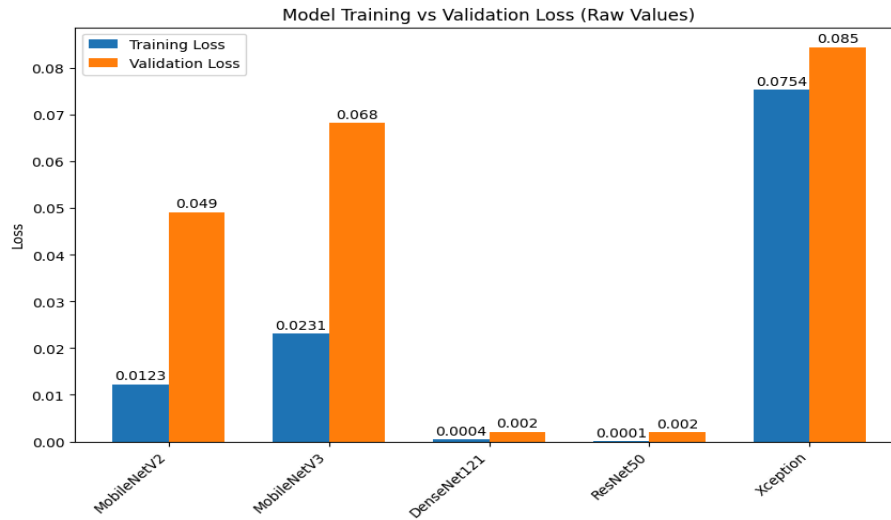


Figure 4.12: Comparison of losses between all models.

In Figure 4.12, a bar chart that compares the final training and validation losses of the five models is provided. DenseNet121 and resnet50 had better convergence characteristics: they had training losses equal to 0.0004 and 0.0000 and validation losses equal to 0.0021 and 0.0020, respectively. This implies that the models were very inaccurate in making predictions. The training loss of MobileNetV 2 was 0.0123 and a validation loss was 0.0491. The training loss of MobileNetV3 was 0.0231 and the validation loss was 0.0682. They were both stable to learn and could be deployed on a lightweight basis. The training and validation losses of Xception were the biggest, 0.0754, and 0.0845, respectively. This implies that it committed larger errors in making predictions. On the whole, the best performance in terms of error minimization was demonstrated by DenseNet121 and ResNet50, and the loss was moderate in the case of MobileNet models but still efficient, whereas Xception had higher loss and still proved to be effective.

4.2.5: Explainable AI Implementation

The research employed the explainable AI (XAI) to make the pest classification models more transparent and easy to comprehend, in such a way that technical and non-technical users can have access to the way decisions are made. XAI can be used to verify whether a deep learning model is examining the correct elements of a given image prior to making a prediction since a deep learning model can be regarded as a black box. This is more important in agriculture where giving inaccurate information may damage the crops and render farmers incapable of making decisions. To accomplish this, the best-performing lightweight model, MobileNetV2 was employed using the Grad-CAM (Gradient-weighted Class Activation Mapping) since it was highly accurate and functioned under resource-limited conditions in real-time. Grad-CAM produces heatmaps which represent the areas of an image that contributed the most to the final classification. The system displays the prediction of the model on leaf textures, color changes, pest marks and infected areas by superimposing these heatmaps over the original pest images.

The displayed visual clarifications shown in Figure 4.13, indicated that the MobileNetV2 was

always selective to features that were biological and significant, such as pest lesions, discoloration, and damaged tissue. This confirms to us that the model has been learning the correct patterns and not merely using random background noise and cues. A XAI addition further increases the reliability of the model, as well as it creates trust in the users. This assists farmers and other experts in the agricultural sector in knowing how a particular group of pests was selected. This is done to ensure that technology is applied responsibly and minimizes the possibilities of misunderstanding, as well as to ensure that people make superior and more confident decisions during practical farming contexts.

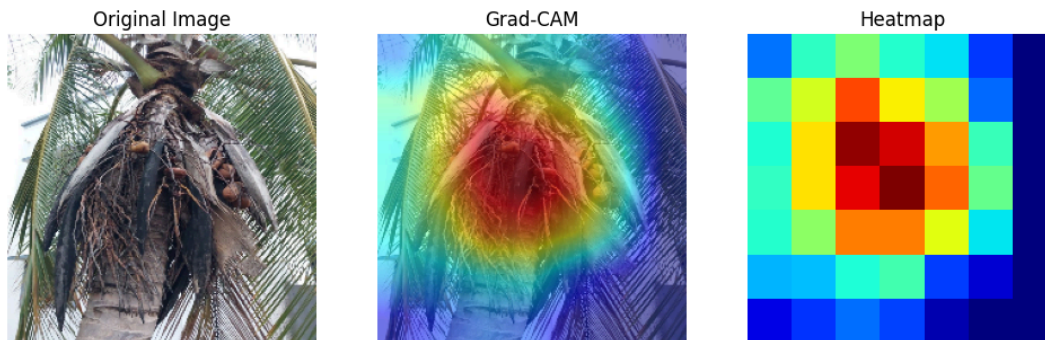


Figure 4.13: Comparison of losses between all models.

4.3 Results and Discussion

The test outcomes reveal that all of the five deep learning models performed very well with the accuracy rates exceeding 97.0 per cent. DenseNet121 and ResNet50, which are heavyweight architectures, achieved perfect or nearly perfect performance with 100% training accuracy and 100% validation accuracy due to perfect precision, recall and F1-scores. It demonstrates that they may identify extremely distinctive aspects in the diseases which appear to be comparable such as Leaf Rot and Yellow Leaf Disease. However, such models require additional computation and extended training periods and, therefore, are not as applicable to on-device applications or in real-time. Lightweight models, such as MobileNetV2 and MobileNet V3, achieved competitive results but were much cheaper to operate. The most promising lightweight model was MobileNetV2 whose training and validation accuracy were 99.8 and 98.7 respectively. Its metrics of evaluation were 0.99 precision, 0.99 recall and 0.99 F1-score always. Its confusion matrix indicates that there are only several misclassification cases of samples and this indicates that MobileNetV2 is suitable in all the five illness classes. MobileNetV3 also performed well with its validation accuracy of 97.60%. It was, however, marginally less accurate than MobileNetV2. Xception performed quite well with the validation accuracy of 98.69 and the accuracy and loss curve were stable with good generalization. Official MobileNetV2 was the most optimal model since it was the most accurate, generalizable, and efficient with respect to computation power. It was the best to use in agriculture since it was capable of capturing important disease features and the inference time was low. Due to such an advantage and its robust and reliable assessment scores,

MobileNetV2 has been selected as the implementation of Grad-CAM. The choice of the model was clear and according to the real symptoms in the pictures 4.14 and 4.15 visualization the parts which were affected by the disease were evident and complimented the real symptoms. This is because, as demonstrated in the results, MobileNetV2 is not only reliable, but it can be employed in early detection systems that are operable in the field, particularly in low resource settings. Figure 4.14 and 4.15 shows the website for coconut disease prediction.

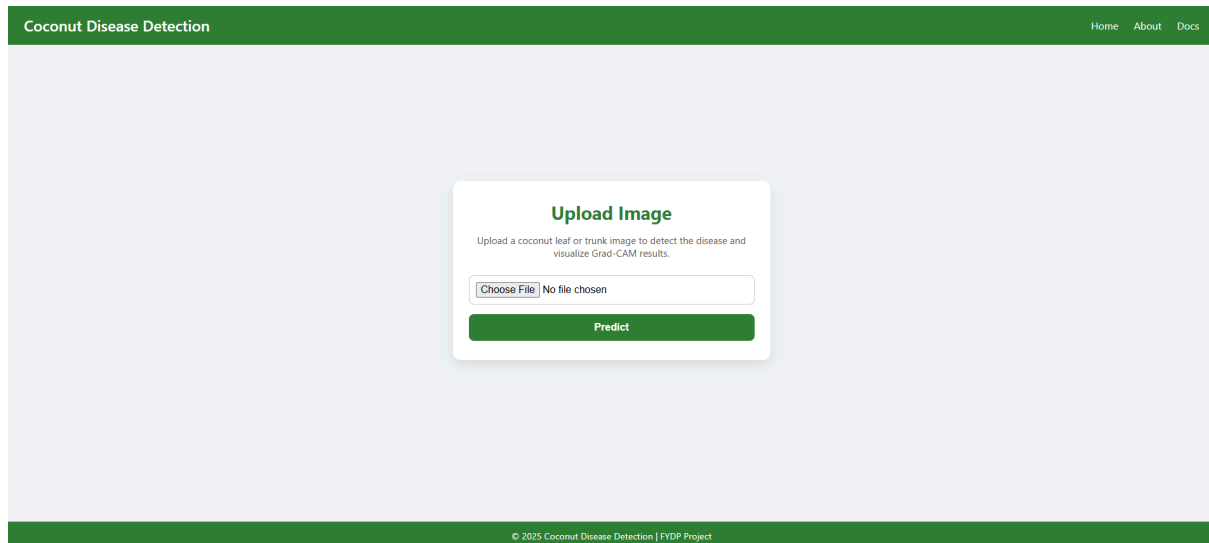


Figure 4.14: Web application home page.

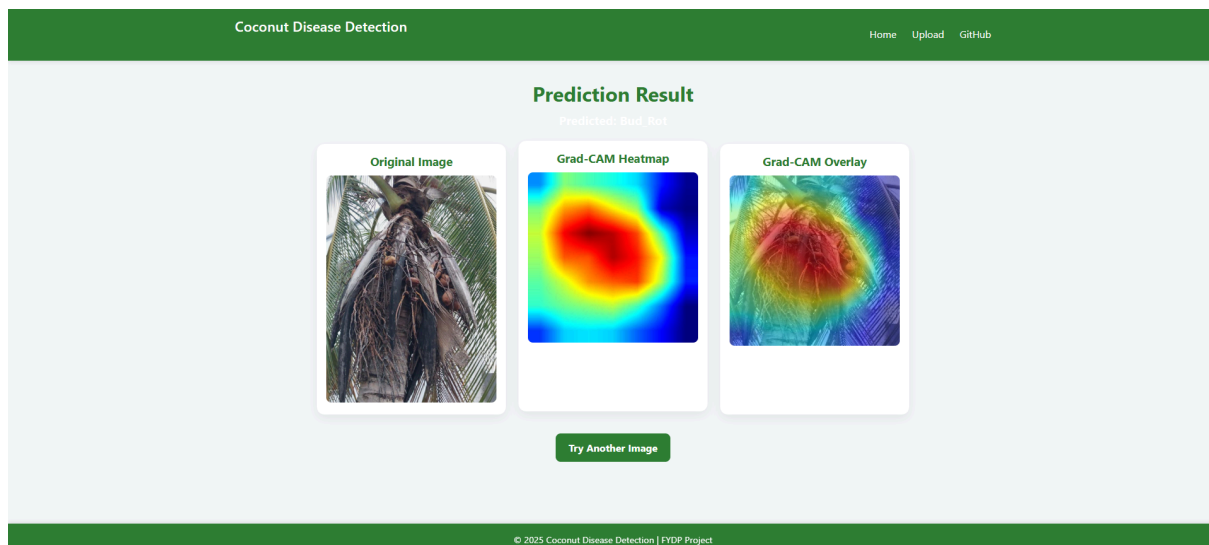


Figure 4.15: Web application result page.

4.4 Summary

This chapter addressed the issue of using, testing and measuring the performance of some deep learning models to classify coconuts diseases. In order to ensure that the findings were reproducible, the environment configuration, preprocessing pipeline and training parameters were described. Five models MobileNet V2, MobileNet V3, and DenseNet121, ResNet50, and Xception were trained and tested in the same manner. Accuracy, loss curves, precision, recall, F1-score, and confusion matrices were used to measure their performance. All the models were found to be effective in the results of the experiment. DenseNet121 and ResNet50 scored nearly perfect and the lightweight models were extremely efficient. MobileNetV2 was the most balanced one in terms of accuracy, as well as the cost of the computation, hence it was a good choice to be used in reality. The application of Grad-CAM proved the interpretability of the selected model, which gives visual support on the way the model identifies disease-related areas. The chapter concludes with the statement that deep learning is quite effective in detecting coconut diseases early and correctly and that lightweight architectures can be employed in the field.

Chapter 5

Engineering Standards and Design Challenges

This chapter gives an overview of the engineering standards and practices and design factors that were involved in making the coconut disease classification system. It discusses software, hardware, and communication standards that were utilized in the project. It also explains the largest design issues that arose in case of working with deep learning models, image data, and less resourceful deployment. The chapter is closed by the analyses of the long-term impacts of the system, its feasibility regarding society, environment, sustainability, ethics, and finances.

5.1 Compliance with the Standards

Best practices in deep learning, computer vision, dataset preprocessing and optimization of models were applied in building the coconut disease detection system. Data were carefully prepared by using their uniform resizing, normalization, and augmentation methods to fit the input demands of all CNN architectures used during the research, such as DenseNet121, ResNet50, MobileNetV2, MobileNetV3, and Xception. These were preprocessing steps that ensured that the images were similar regardless of their light status, resolution and the background they had before they reached the models. The assessment procedure was based on common measures of performance such as accuracy, precision, recall, F1-score and the training-validation loss curve hence it was quite simple to compare all the models. The training configurations were best practice such as, applying early stopping and regulated modifications of the learning rate to prevent overfitting. Ethical and operational standards were also adhered to in the project since it ensured that the dataset was accessible to all and did not have any confidential data. MobileNetV2 was decided upon as the final model to be deployed in the industry since it fulfills the industry requirement of having an accurate and computationally efficient model that could be used in real-life scenarios which depend on mobile devices. The limitations of dataset diversity, visual fusion of disease types, class imbalance and model complexity were addressed by augmentation, regularization, transfer learning and selecting both fast and accurate architectures. All in all, the engineering standards of the system emphasizing accuracy, reliability, fairness, maintainability, and capability to be applicable in the real world were still met.

5.1.1 Software Standards

The project adopted the general standards of software used in the development and testing stage. The primary programming language was Python since it can be read easily and because

it provides a robust machine learning research ecosystem. These models were constructed, trained, and evaluated with frameworks, such as TensorFlow and Keras, that provided GPUs with acceleration, a high-level API, and excessive community support. These frameworks ensured that the models could be applicable in similar ways and be replicated in other environments. Math, data clean-up, and data conversion of arrays were easy to perform in libraries such as NumPy and Pandas and were all of relevance to large image datasets.

To ensure accuracy, loss, confusion matrices and Grad-CAM heatmaps, we made graphs using visualization tools such as Matplotlib. In larger networks such as DenseNet121 and Xception, CUDA and cuDNN ensured that training was quicker on the workstation. Git and GitHub were used to track the changes in various phases of the experimentation and it was also much easier to maintain clear records of model modification, dataset modifications and training processes. These software standards ensured that the project remained steady, maintainable and scalable and it allowed people to collaborate and improve in the future.

5.1.2 Hardware Standards

The hardware setting was sufficient to train and test deep learning models according to the performance standard requirements. An architecture requiring a GPU was required to match the computing requirements of CNN architectures and, in particular, DenseNet121 and ResNet50, with millions of parameters. T4 Google Colab GPU accelerated training, reduced the time spent on calculations, and made it possible to train larger batches. Adequate RAM also ensured that data loading, pipelines to augment data, and model training proceeded well without any form of slowness.

Storage criteria were also considered since the dataset, model checkpoints, logs and output visualizations were all very memory intensive. The speedy and dependable storage was important, as that way the checkpoints could be saved and the training could be retrieved where it was previously interrupted in case it was necessary. This hardware architecture was consistent with generally accepted standards of machine learning experimentation, which implied that there is consistency in model training, testing, and comparison, making all three rates fast and reliable.

5.1.3 Communication Standards

The communication and collaboration norms were relevant so as to control the flow of work in the project, to monitor the experiments, and to prepare the system to be potentially integrated into the mobile or web platform. People primarily communicated with each other and managed the code using GitHub. It allowed tracking all changes, tracking of issues, keeping track of all changes, and tracking of all changes. The experiments were presented in an interactive manner using Google Colab notebooks. All the code, results, graphs and explanations were in one place in each notebook and therefore could be shared with fellow workers or superiors.

The system is based on common communication standards such as REST API compatibility, data models in the form of JSON and secure connection in HTTPS. This renders easy

incorporation into mobile applications or cloud-based solutions in the future. These communication standards ease maintenance and addition of the system since it becomes clear, traceable and organized.

5.2 Impact on Society, Environment and Sustainability

The coconut disease detection system would significantly enhance the well being of the environment, the people and agriculture. Provided that they can recognize the diseases early and properly, farmers are able to intervene and prevent the extension of such diseases as Bud Rot, Yellow Leaf Disease, and Leaf Rot. This would save the destruction of plantations. Early detection of problems does not require so much pesticides which is healthy to the environment and your health also. The reduced use of pesticides ensures that the soil is less polluted, the sources are less toxic, and the crops are more healthy.

It is a helpful system to be used by small-scale farmers who may not necessarily have access to agricultural specialists or laboratory examination. The system provides rural farmers with the ability to make intelligent decisions because they can easily and correctly diagnose diseases through their smartphones. This assists in stabilizing incomes, reduces losses of crops and food is more secure in the region. Sustainability wise, the system promotes precision farming, that is, only the treatments and interventions that are required are applied. This reduces wastage, pollution and assists in maintaining the environment in balance in the long run.

5.2.1 Impact on Life

It is significant that the system has its impacts on improving the life of people particularly in rural farming areas. By providing them with an inexpensive and convenient diagnosis tool, farmers will be able to identify the diseases early enough to prevent massive losses in production. This not only secures their income, but also assists the agricultural economy to be stable in regions where coconut farming is a key source of income. Since it is compatible with lightweight models such as MobileNet V2, it can be used in standard smart phones. This implies that it can be used by even the farmers who are not in a position to purchase more advanced devices.

The reduced use of the unnecessary pesticides is also beneficial to the health of people. There is also a reduction of the risks of acquiring long-term health issues because the farmers, families, and people living around them are less likely to be exposed to harmful chemicals. Food is safer to consume because of healthier crops and reduces chances of contamination. It is a system that makes life better as it provides a safer environment, more stable economy and healthier life, which makes the life of farming an easier and more reliable way of life.

5.2.2 Impact on Society & Environment

Its system assists in sustainable development of agriculture on a greater scale, introducing modern AI-based instruments into the past farming practices. It also makes disease control more precise and data-driven, reducing trial-and-error that can result in unnecessary crop destruction or excessive chemical application. By doing so, food would be more accessible,

the local economies would be assisted, and the farming support services will find it easy to perform their duties. The impact on the environment is similarly large. With the reduction of using pesticides, we can preserve the fertility of the soil, maintain biodiversity, and prevent the degradation of the groundwater and other surrounding ecosystems. Some of these organisms are bees, earthworms and other useful insects which maintain a balance of the ecology. They are more comfortable when they are not subjected to as many chemicals. Thus, the system is assisting in farming and environmental protection.

5.2.3 Ethical Aspects

Ethics had to be considered when developing and implementing the system. The training dataset was based on publicly available repositories, thus there was no confidential or sensitive information in it. Fairness and neutrality was maintained through balancing the dataset and applying augmentation and oversampling to decrease bias. With Grad-CAM visualizations, the farmers could visualize why the model had predicted a particular disease and thus deep learning was not as much of a black box problem. To minimize the energy consumption, we selected lightweight architectures to be used, and this choice contributed to the reduction of the environmental footprint of AI models. The system was designed in a way that it is simple to operate and affordable so that small farmers are not left behind as they are unable to afford it. These ethical issues ensure that the system remains accountable, just and beneficial to the communities in which it is intended to assist.

5.2.4 Sustainability Plan

The sustainability plan will involve constructing a disease detection system which is long lived, flexible, and low-cost. The system is capable of performing well on smartphones without a need to have high-end hardware due to its lightweight architectures such as MobileNet V2. This renders it ideal to rural and remote locations. Using cloud-based configurations, the users do not need to retrain or reinstall anything when new disease images are being availed. This maintains the accuracy of the models with time. The system may be included in mobile applications or farm advisory services to earn money on subscriptions, partnerships, or institutional license. This will contribute towards the economy remaining robust. The support system in various languages, and easy to use interfaces allow the farmers to use the technology easily, which contributes to social sustainability. In order to save the environment, we should utilize fewer chemicals and promote more specific agricultural practices. This plan ensures that the system remains helpful, significant and relevant over a long period of time.

5.3 Project Management and Financial Analysis

The project has been structured into phases including data collection, data cleaning, model training, testing and preparation to be utilized. A low-budget and a high-budget approach is considered in the financial analysis. A low budget involves the implementation of free services such as Google Colab and already-owned equipment, which is acceptable at school or small projects. The extended budget has specific GPUs, paid cloud, app development on the mobile platform, and long term storage. This

renders it ideal to be used by the government or businesses. Mobile apps, service-based pricing to farmers, institutional licensing to agricultural departments, and alliances with agricultural research centers are all viable revenue models. These financial alternatives are used to ensure that the project remains viable and can develop over the span, in addition to providing farmers an access to low cost and dependable technology.

5.3.1 Low-Cost Research Setup

This project has utilized free platforms and the use of open-source software such as Google Drive, Github, and Colab Notebook to train and store. This is how one can conduct research in academics. Nonetheless, some minor expenses were also involved in this project. Estimated budget analysis shown in Table 5.1.

Table 5.1: Minimum viable budget estimation

Item	Cost (USD)
GPU-(Colab T4)	50 \$
Data Storage & Backup	10 \$
Software	10 \$
Internet & Utilities	30 \$
Others	30 \$
Total Estimated Budget	130 \$

5.3.2 Advanced Computational Setup

The system will be expensive to scale to high volume training, commercially or long-term research. A high-level computation system will require high-performance hardware, software, and other expenses. Estimated budget analysis for advanced computational setup shown in Table 5.1.

Table 5.2: Advanced computational setup budget estimation

Item	Cost (USD)
High Performance GPU	1000 - 1200 \$
High-performance workstation	2000 - 2500 \$
Storage & Backup	1500 - 2000 \$
Internet & Utilities	100 - 200 \$

Others	200 - 500 \$
Total Estimated Budget	4000 - 6000 \$

5.3.3 Potential Funding Sources

The proposed coconut disease detection system can be funded in many ways, which are effective and practical. Users can get the project money in several major ways and all of them will contribute to the project making it possible, scalable, and accessible to the farming communities.

5.3.3.1 Government Funding

The government can provide huge amounts of money in the form of digital agriculture, smart farming, and rural development programs. Such projects tend to be in favor of technologies that enhance productivity, simplify food, and have a lesser impact on the environment. This qualifies this project as a viable one when it comes to agricultural innovation grants and subsidies.

5.3.3.2 Private Sector Sponsorship

Money and technology can be assisted by working with companies that produce farm equipment, agricultural technology or agribusiness. These companies tend to purchase the equipment that will assist them in monitoring their crops and addressing diseases. The reason is that these systems align with their long-term business strategies and objectives of innovation.

5.3.3.3 Crowdfunding

One of the methods to raise money is through crowdfunding among the local population, in particular, those who are interested in sustainable farming and innovative technologies. The level of interest that people have in utilizing AI solutions in farming can make enough money to finance the development process.

5.3.3.4 Non-profit Organizations

This energy project can be funded by NGOs that deal with food security, rural development and poverty reduction since it will directly assist farmers to earn more and increase their crop production. They tend to support low cost-high impact technological solutions applicable in communities that lack sufficient resources.

5.3.4 Return on Investment (ROI)

The payback of this system in short-term and long-term to society and the environment can be looked at. The farmers will have a clear explanation on how the system will enable them to make more money. The project is not only money making, but it possesses numerous other advantages. It assists in making farming more sustainable, promotes the adoption of new

technologies in farming, and assists in the long-term food security. The cost-effectiveness (ROI) is influenced by the number of individuals utilizing the system, the frequency with which it can be integrated into other regions, and its ability to integrate with the local agriculture procedures. The net impact on the productivity, the health of the environment and the well being of farmers is very large when used in most communities. These merits of this system will probably surpass the expenses of developing and establishing it. The reason is that it will result in a long-time production of crop yields, reduced use of chemicals, and increased sustainability in general. The system is an excellent long-term investment between the partners and beneficiaries provided it receives the appropriate money, the appropriate resources, and develops at a slow rate.

5.3.4.1 Better Pest Management

It is through the system that diseases that affect coconut trees can be discovered before they spread, and this significantly reduces loss of crops, as well as increases the annual production. An example is of a farmer with crops valued at 5,000 dollars who will earn an extra 500 to 800 dollars annually in total by preventing disease transmission. Reduction in the use of pesticides also reduces the cost of production and the food is also improved.

5.3.4.2 Technological, Environmental, and Social Gains

The long-term benefits of the system to the environment are the reduced number of chemicals, healthier soil, and safer food to grow. It encourages the adoption of new farming technologies, aids in the protection of biodiversity, and improves food security in general. Long-term ROI presupposes that many people use it, it will be possible to scale it in other regions, and agriculture will become more productive in general.

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

Complex engineering issues that were handled by this project required individuals with varying degrees of technical expertise, analytical abilities, and practical design expertise to collaborate. The type of diseases that attack the coconut leaves is quite difficult to know. The reason is that the pictures could be modified, there are excessive classes, and the equipment is not very high in quality. It is not an easy issue to solve since it needs much knowledge about the deep learning structures, cautious pre-processing plans, contrasting among the models and considering how to execute it in practice. Other ways of solving the problem are also picking the correct models, achieving the correct balance between cost and accuracy, having limited data to work with and ensuring that Grad-CAM makes sense. These sections indicate that the project is a more challenging engineering problem that involves more than just technical expertise and a methodology of decision-making that is coherent. Table 5.3 shows the mapping with complex engineering problems.

Table 5.3: Mapping with Complex Engineering Problem.

EP1 Dept of Knowledge	EP2 Range Of Conflicting Requirements	EP3 Depth of Analyses	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent Of Stake- holder Involvement	EP7 Interdependence
✓	✓	✓	✓			✓

Justifications:

- **EP1 – Depth of Knowledge**

Solid knowledge in deep learning, computer vision, and CNN-based architectures were needed in the project. The use of transfer learning, data augmentation, class balancing, and model fine-tuning required close knowledge of the concept of TensorFlow/Keras, training with the aid of a GPU, and evaluation metrics, including accuracy, loss, precision, recall, and F1-score. Further insights into Grad-CAM were needed to generate visual explanations thus the system is interpretable and can be practically implemented in the diagnosis of coconut disease. [ch-3].

- **EP2 – Range of Conflicting Requirements**

The system needed to be extremely precise and must be light enough to operate in real time particularly on mobile devices and other hardware that has very few resources. In the cases when we tested the heavy models (DenseNet121, ResNet50) and light models (MobileNetV2, MobileNetV3), we had to compromise between performance and cost. An uneven dataset also had to be addressed by the model, data use made ethical, and the model also had to perform well across a wide range of field conditions, such as lighting alteration and noise in leaf images. [ch-4] [4.1, 4.2]

- **EP3 – Depth of Analysis**

Each of the five models was thoroughly explored, including accuracy/loss curves, confusion matrices, and precision, recall and F1-scores of new classes. Comparative analysis revealed that computing has trade-offs between speed and accuracy. The model that we selected as the most effective is MobileNetV2, which has a high accuracy rate (98.7%) and has a lightweight architecture as indicated by extensive quantitative and visual analyses. [ch-4] [4.2.4]

- **EP4 – Familiarity of Issues**

The project addressed issues of typical agricultural image datasets, such as, poor image quality, noise, background clutter, imbalance in classes and mislabeled samples. In order to solve these issues, we applied several preprocessing techniques, including cleaning, resizing, normalization, augmentation, and class balancing. The system was enhanced such that it would be applicable in real life where coconut leaves may appear to be dishevelled, half visible, or trapped in poor light. [ch-3] [3.2.2]

- **EP7 – Interdependence**

The project had numerous components, which were interdependent, including data preprocessing, model training, transfer learning, augmentation pipelines, evaluation frameworks, and explainability (Grad-CAM). Each of the modules relied on the output of another. As an example, balanced samples helped models to become more stable and trained CNNs helped to make Grad-CAM visualization more valid. Such a linked workflow has ensured the existence of a disease detection system that is well-coordinated and in a single location. [ch-3] [3.2]

Mapping with Knowledge Profile

Table 5.4: Mapping with knowledge Profile.

K1 Natur al Scien ce	K2 Mathema tics	K3 Engineerin g Fundamen tals	K4 Speciali st Knowle dge	K5 Engineer ing Design	K6 Engineer ing Practice	K7 Comprehe nsion	K8 Resear ch Literatu re
		✓	✓	✓	✓	✓	✓

Justifications:

- **K3 – Engineering Fundamentals**

The project applied simple engineering concepts regarding machine learning, optimization, and evaluation. The theoretical foundations of training and testing models were concepts such as convolution, activation functions, control over overfitting, loss minimization, and statistical measurements of accuracy. A simple knowledge of engineering was also necessary to compare CNN-based architectures (DenseNet121, ResNet50, MobileNetV2, MobileNetV3 and Xception) and determine what they excelled at and what they were not good at. [Ch. 3, Ch. 4]

- **K4 – Specialist Knowledge**

The development of the system needs high expertise in deep learning, particularly CNNs, transfer learning, and explainable AI. In order to create a good coconut disease detection model, we must know about model architectures, feature extraction, class balancing, data augmentation, and Grad-CAM. This is a deeper understanding of ML and will demonstrate that the person has an understanding of computer vision in agriculture. [Ch3] [3.2, 3.2.9]

- **K5 – Engineering Design**

The whole workflow, including preprocessing data, the selection of an augmentation strategy, the choice of a model, and the configuration of training and its evaluation, had to consider both the functional and non-functional considerations. Through

accuracy optimization, minimized work of the computer, and ensured that the model could be employed in real life context such as on mobile devices or on the edge. The design decisions considered speed, the use of resources, ease to understand and the reliability of the classification. [Ch-3] [3.1, 3.2]

- **K6 – Engineering Practice**

The project adhered to good engineering standards such as data cleaning, ensuring that the process of training could be repeated, comparing training models in a methodical manner, version tracking, and structured report writing. The contemporary engineering processes, such as model experimentation, fine-tuning, evaluation based on metrics, and Grad-CAM visualization are all present in the real-life AI deployments. It was also reflected in the work on how to do such things as keep records and try things out multiple times. [ch-4] [4.1, 4.2, 4.2.4, 4.2.5].

- **K8 – Research Literature**

The methodology was shaped by a comprehensive analysis of the past research on the problem of plant disease detection, CNN architectures, transfer learning, and explainable AI. The literature review was useful in identifying gaps in the literature, including imbalance in classes, absence of lightweight models and interpretability. [Ch-2] [2.2, 2.3]

5.4.2 Engineering Activities

Some of the major engineering tasks in the project included the preparation of datasets, model training, systematic testing, as well as how to design an explainable deep learning pipeline to detect coconut disease. The correct choice of resources (EA1), collaborating and being supervised during development (EA2), and a responsible amalgamation of multiple CNN models, with the Grad-CAM explainability, into an efficient workflow proved to be important. The environmental and social impact was also considered in the project such as the reduction of the use of pesticides and encouraging environmentally friendly farming (EA4). Knowledge of machine learning tools and prior experience in similar courses enabled one to experiment with them despite having limited hardware (EA5). Overall, there is a positive balance of technical skill, responsible design, and knowledge of the real world in the work.

Mapping with Complex Engineering Activities

Table 5.5: Mapping with Complex Engineering Activities.

EA1 Range of re- sources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for society and environment	EA5 Familiarity
✓	✓		✓	✓

Justifications:

- **EA1 – Range of Resources**

A lot of tools and resources were utilized in the project, such as TensorFlow/Keras, a GPU-based training tool, Google Colab, version control tool, Grad-CAM and Matplotlib visualization tools. Libraries were also used to process images, augmentation modules, and multiple CNN architectures were used in the project. This broad spectrum of digital and computational means demonstrates that there is much engineering involved. [ch-4] [4.1, 4.1.1, 4.1.2].

- **EA2 – Level of Interaction**

Discussion with peers, academic supervisors, and understanding of finding diseases in crops all impacted the system design. This demonstrates engagement with various views and this is a main demand of EA2.

- **EA4 – Consequences for Society and Environment**

The proposed detection system promotes sustainable agriculture by reducing the use of unnecessary pesticides, preserving crop yield and assisting in maintenance of food security. It reduces pollution and improves the soil and water through reducing reliance of people on chemicals. Earlier diagnosis of the diseases saves the farmers time loss of crops, and the quality of life generally increases. All these are good to the people and the environment.

- **EA5 – Familiarity**

We also had to operate within realistic constraints such as a limited size of data, the computational constraints of the free tier of Colab, varying image quality and the tradeoff between heavy and light architectures. Owing to your prior knowledge of machine learning workflows and prior classes on machine learning workflows, your prior experience with CNNs and transfer learning, you were able to conceive prototypes quickly and experiment with new concepts. This continued to drive the project despite the low resources. [Ch-3] [3.3], [ch-4] [4.1]

5.5 Summary

This chapter talked about how the coconut disease detection system, which uses deep learning according to engineering requirements, design considerations, and real-world implications. It discussed how the project adhered to the set standards of software hardware as well as communication to ensure that the system remains reliable, efficient, and scalable in various deployment environments. The models and training strategies that were chosen followed the principles of machine learning, including the organization of preprocessing, the evaluation of performance and optimization, aimed at increasing its accuracy without affecting the costs of calculations. The chapter also discussed the ways in which the system would benefit the society in significant ways particularly to smallholder farmers who in most cases do not have access to sophisticated diagnostic measures. The system assists in making food healthier, the economic system more stable and agriculture more resilient by reducing the use of unnecessary pesticides, identifying diseases earlier and making crops healthier. Even to minimize harm and establish trust, ethical concerns such as fairness, openness, data privacy and responsible model behavior were discussed. The lightweight model design, low-costs mobile and edge deployment and continual community-based feedback loops keeping the system relevant also led to sustainability. The chapter also discussed some of the largest engineering issues that needed to be addressed which included balance between accuracy and the constrained resources, addressing real time-performance requirements, operating within the constraints of datasets. These discussions demonstrate how the project gets integrated with technical rigor and social responsibility and a long term environmental consciousness.

Chapter 6

Conclusion

This chapter provides a summary of the research results, talks about how important the coconut tree disease detection system is, and describes its flaws and what it could do in the future. It gives an overview of the whole process of designing, training, and testing several deep learning models. It also talks about how the system could help agriculture, especially in areas where resources are limited.

6.1 Summary

This study presented a comprehensive deep learning framework aimed at classifying five categories of coconut tree diseases utilizing a newly assembled dataset from online sources. To improve quality and reduce bias before training, the dataset was carefully cleaned, resized, normalized, and augmented. We trained and tested five well-known convolutional neural networks DenseNet121, ResNet50, MobileNetV2, MobileNetV3, and Xception to find the best one for use in the real world. All of the models did very well, and DenseNet121 and ResNet50 had perfect precision, recall, and F1-scores. MobileNetV2, on the other hand, had the most balanced performance, with 99.8% training accuracy and 98.7% validation accuracy, and it also didn't need a lot of processing power. This made MobileNetV2 the best model for apps that run on mobile devices or at the edge. Grad-CAM was used on MobileNetV2 to make it easier to understand. It created heatmap visualizations that show the areas that affect the decision-making process. This step makes sure that the classification results are clear and trustworthy, especially for people who aren't tech-savvy and work in agriculture. The system as a whole shows a lot of promise for helping farmers, especially those in rural areas with few resources, by making it easier and faster to find diseases caused by pests. Farmers can use fewer pesticides, protect the quality of their coconut yield, and use more environmentally friendly farming methods if they find pests early on. Also taken into account were ethical issues like fairness, data safety, and accessibility. This made sure that the system is responsible, trustworthy, and good for long-term agricultural growth.

6.2 Limitation

Even though the research looks good on paper, there are still some issues that could create problems in the real world implementation. The dataset doesn't have a lot of natural variation in things like lighting, angle, and field conditions. This means that the model might have trouble when it sees real-world images that are very different from the training data. Even though augmentation helped balance the classes, synthetic transformations can't completely replace real field diversity. Also, DenseNet121 and ResNet50 are too heavy for low-power devices because they need a lot of memory and processing power. MobileNetV2 works well most of the time, but it does have some problems with images that are noisy, blurry, or partially blocked. Finally the system may be suitable with light weight mobile

devices but the system also needs clear, close-up photos from the user, which might be hard for farmers who use older phones.

6.3 Future Work

Future development can fix these problems and make the system work better and be easier to use. Adding more real farm pictures from different places, times of year, and weather conditions to the dataset would make the model much more flexible. Using advanced synthetic generation methods like GAN-based augmentation could make new variations and fix class imbalance more effectively. Techniques like quantization, pruning, and lightweight distillation could make MobileNetV2 even smaller and better for smartphones, edge devices, and offline settings without losing accuracy. Farmers would be able to quickly identify pests by uploading a picture or taking a picture with their phone if the model were built into a full-featured mobile app. If the system could work with IoT devices and automated monitoring tools, it could allow for ongoing real-time detection and warning systems for large plantations. Improvements to the user interface, like language options for different areas, voice-guided instructions, and easier ways to understand results, would make it much easier for rural farmers to use and adopt. Finally, the model will stay accurate, reliable, and relevant over time if we keep adding new field data and feedback to it. This is because disease patterns change over time.